Introduction to Designing API's

Helping Developers Succeed

READ the **Westpac Mesh Guide** to understand who and what we are

Then move to the following content for the designer story so you can start to deliver great customer outcomes

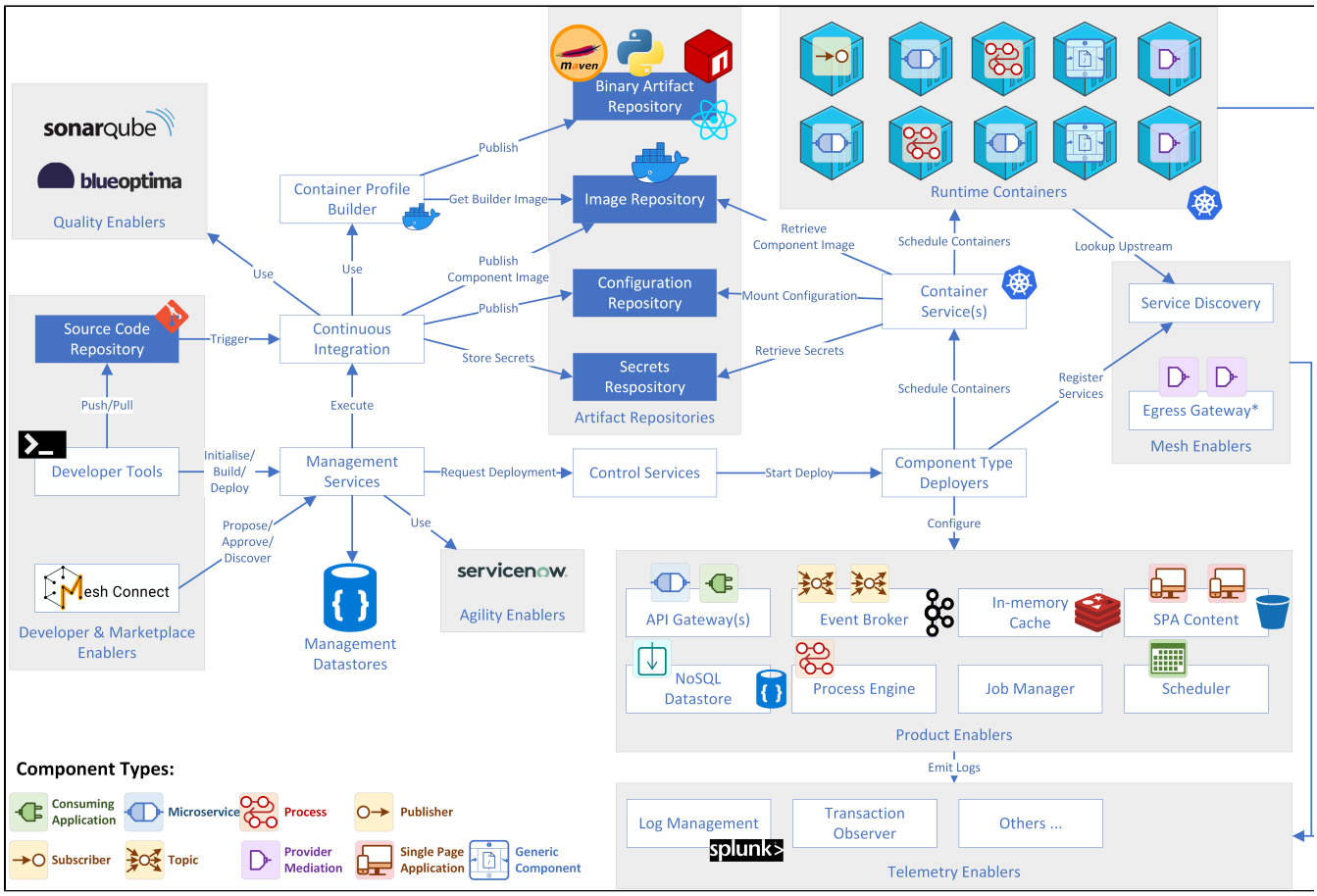
1 Introduction to API Designer

Mesh Platform Overview

The Mesh Platform is realised by a suite of interacting subsystems which have specific roles in providing services to end-users, manufacturing and running Components.

This page will enumerate the coarse-grained subsystems that are required to realise the platform and outline their roles.

Platform Subsystem Architecture Diagram



Subsystem overview

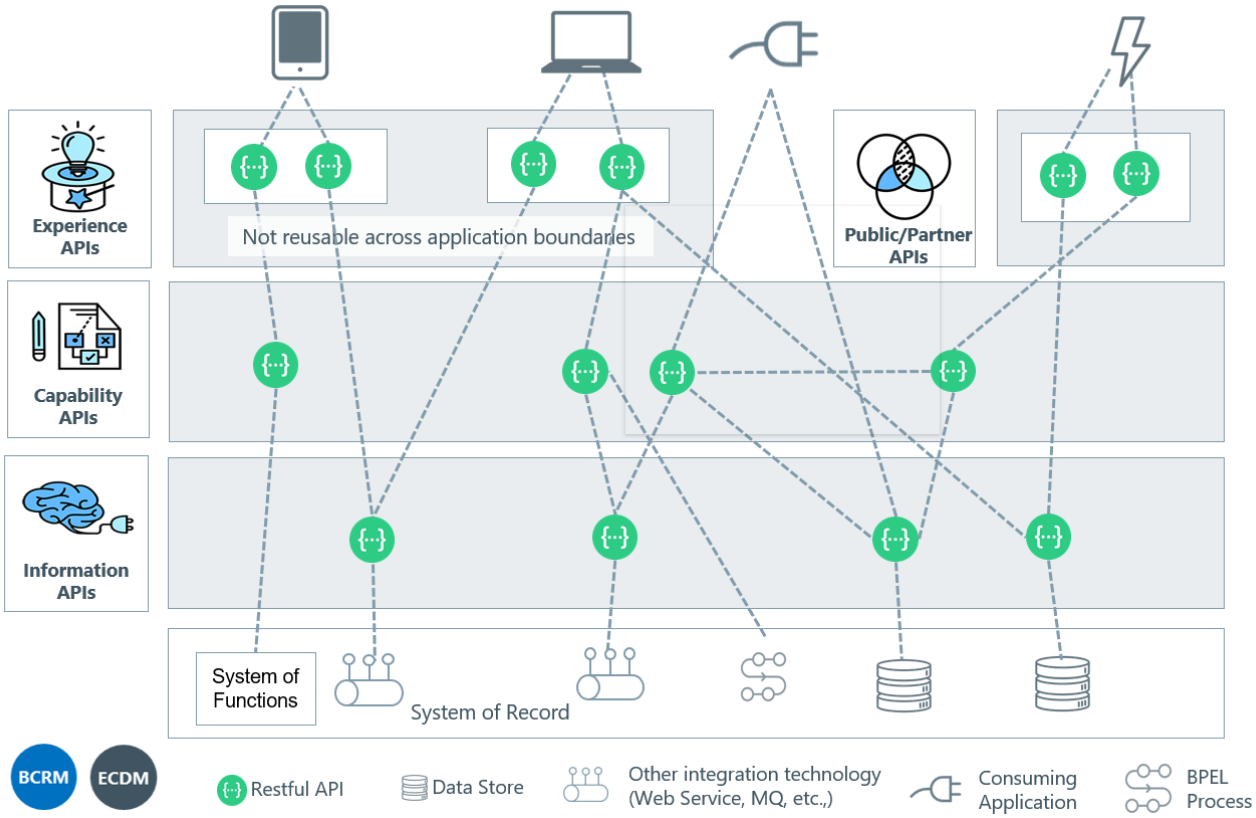
Subsystem	Purpose	Candidate/Actual Technologies
Mesh Enablers		
Container Service	Provides segregated, scalable compute on-demand to run Components	**Azure AKS**, AWS EKS
Service Discovery	Allows Components to find and consume other Components	Istio, **Consul Connect**
Egress Gateway	Mediates calls from platform-hosted Components to off-platform services - sometimes called a Terminating Gateway	Linked to Service Discovery/Service Mesh
Platform Interaction Enablers		
Source Code Repository	Store Component source code history	**BitBucket**, GitHub
Developer Tools	Command-line tools to work with Components and interact with the Platform	**Custom build**, Waypoint
Mesh Connect	Application to discover, propose and manage Component portfolios	**Custom build using Mesh technologies**
Management Services	Support Mesh Connect and Developer Tools and composed automations	**Custom build** w/Datastores
Software Storage, Build & Delivery Enablers		
Continuous Integration	Support manufacturing, validating and publishing Component Versions from source	**Bamboo**, Jenkins
Control Services	Support deploying/undeploying Component Versions from binary repositories and getting 'state of the world'	**Microservices + Bamboo**, Jenkins, Azure /AWS services
Container Profile Builder	A versioned, containerised stack of build tools to predictably manufacture binary artifacts for a Component	**Podman, Buildah, Docker, Maven, OpenJDK, NPM**

Component Type Deployers	A versioned, containerised stack of deploy tools to predictably deploy Component versions to an environment	**Kubernetes + Bamboo**, Jenkins, Azure /AWS services
Binary Artifact Repository	Polyglot repository for storing deployable artifacts of Components indefinitely	**Artifactory**, Nexus
Image Repository	Docker Image repository for storing runnable images of Components and Platform toolchains	**Artifactory**, Docker Registry, Azure Container Registry, Amazon ECR
Configuration Repository	Stores general and environment-specific configuration settings for Components	**Artifactory**, Database
Secrets Repository	Stores sensitive environment-specific configuration settings for Components	**Artifactory**, Vault, Azure Key-Vault
Product Enablers		
API Gateway	Traditional Ingress Gateway for APIs enabling off-Mesh systems /applications to invoke APIs	Axway, **Kong**, Tyk, Azure API Management, AWS API Gateway
Event Broker	Run secure Pub/Sub topics	**Kafka**
In-memory Cache	Improve latency issues with slow/low-volume upstream systems by enabling an in-memory cache	**Redis**
Single Page Application Hosting	Serve up SPA static content and provide security integration to IdP(s)	**AWS S3 & CloudFront**, Kubernetes, OBMR
NoSQL Datastore	Provide persistent datastores for microservices in a Data Mesh	**Cassandra**, MongoDB, CockroachDB
Telemetry Enablers		
Log Analysis	System to store all logs from Components & subsystems suitable for large scale analysis	**Splunk>**
Transaction Observer	Provide visibility of transactions across components in real-time	App Dynamics, Zipkin, Splunk Observability

Subsystem Service Quality Attributes

Further reading: Westpac Mesh for Architects

Multi-Speed API Layering



When designing your solution, divide your components into the following distinct layers:

1. Micro Frontends / Experience API - Micro Frontend components combines both user-interface and Experience API code bases in a single repository. Experience APIs are tightly coupled to the user-interface design, should not be reusable, and should be thrown away when the user-interface they are coupled with is removed. Experience APIs can use capability and information APIs to provide the necessary functionality.
2. Partner APIs - Partner APIs expose our products and services to companies that we have an established relationship with. Partner APIs use capability and information APIs to provide the necessary functionality. Partner APIs are used by another companies application.
3. Public APIs - Public APIs can be called by entities that may have no direct or indirect relationship with the bank and expose only publicly available information. For example ATM locations, branch locations etc. **Note that Public APIs cannot call private (Capability, Information APIs)**
4. Capability APIs - These APIs encapsulate business processes, business rules etc. These APIs typically call one or more information APIs and are reused across channels. This layer may contain state accessed via an API. For example, a capability API encapsulating a business process would require the current step of the process to be stored.
5. Information APIs - The APIs perform CRUDS (create, read, update, delete and search) operations on entities. For example, retrieving and updating customer details. These APIs are called by partner, experience and capability APIs and are typically highly reusable. Access to the entity is ONLY allowed via an API. Note that Information APIs ONLY return/accept canonical values for reference data - please refer to Pattern 7: Reference Data Cache Pattern.
    a. Information APIs are split into two physical APIs. One of the APIs does read and search, the other does create, update and delete. This aligns with the CQRS Pattern . The naming convention for Information APIs outlines how the APIs are named and has some more information. Major reasons we considered for this decision are ,
        i. **Security** – Having read and write separate APIs allows to set access permission properly . If we leave the write operations in the same APIs, all the read consumers will also get access to those.
        ii. **Scalability** - Read APIs require more TPS , so separating them helps to scale them better
        iii. **Different Cadences for Build** - Read APIs are reused and extended heavily. If the write operations are combined into same API, all those endpoints require regression testing.
    b. Information API advocated positions.

<u>API Ownership</u>

An API belongs to a Domain.

- The Domain is aligned with Line of Business.
- The Domain Architect is responsible for managing new components within their domain, and any technical debt for existing components.
- The Domain Owner represents the people responsible for maintaining and providing ongoing support for the API on Westpac Mesh. The Domain Owner and Delegates are responsible for approving, managing support teams and development teams for their components.

Domain Owner

Every API within Westpac goes through a lifecycle. It is the Owners responsibility to ensure that this lifecycle is managed to ensure the impact to consumers of the API is minimised.

1. Certificate management - The Domain Owner is responsible for the ordering of the certificate for an API.
2. Capacity Management - TPS , rate limit etc . It is the owners responsibility to ensure that both these TPS limits are being set appropriately.
3. If these are not provided by the owner, the platform will **default to 5TPS** for all capacity limits. That is, the API will be limited to 5TPS and each consumer will be limited to 5TPS.
4. Quality Management - the owner is responsible for the quality of the APIs being produced
5. Consumption Management
6. Support Management

Domain Architect

Solution Architects are responsible for delivering a solution design to meet a business need. This will often take them across domains such as customer, payments, group operations etc. The risk is that with their focus on designing a solution, an incoherent set of APIs is delivered from an organisation perspective,. For example, APIs that duplicate functionality.

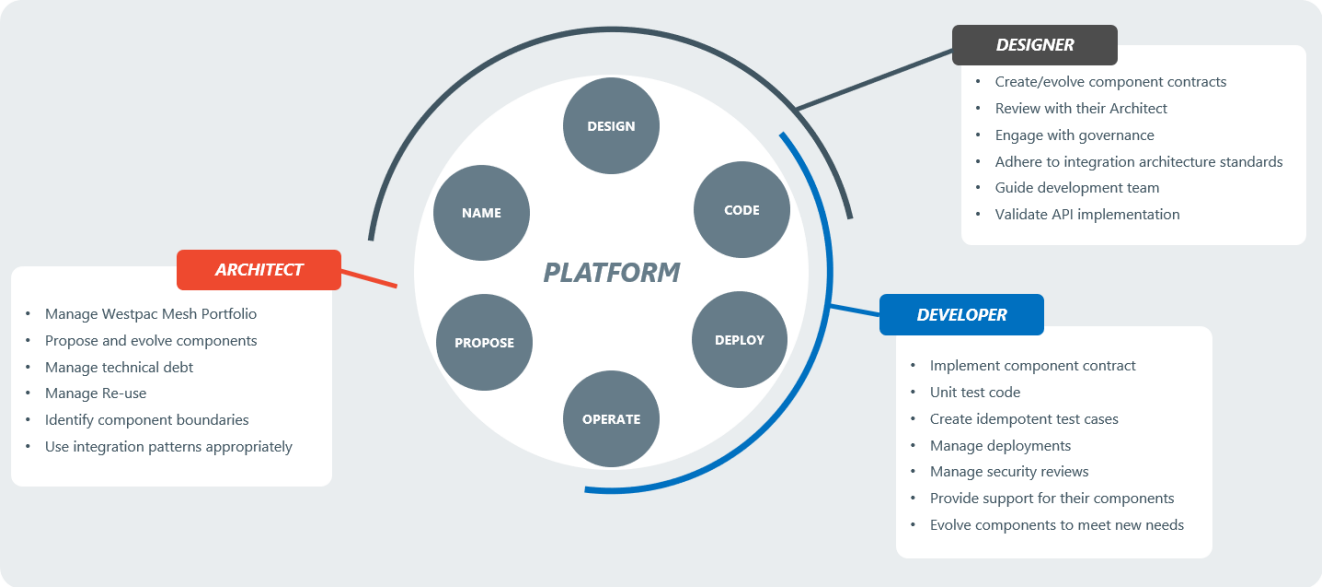
To mitigate this risk, the Domain Architect(s) work with the Solution Architect to ensure a solution that aligns with the organisations direction and meets the business need.

Domain Architect's primary roles are ,

- Ensures that changes to existing APIs are architecturally sound.
- Ensures that new APIs are architecturally sound and belong to their domain.
- Ensures that the API Portfolio for a domain is consistent.
- Ensures that technical debt within the domain API portfolio is being actively managed.

Your role as a Westpac Mesh Designer

Overview



Your role as a Westpac Mesh Designer

API designers are responsible for completing the details of the high level API design provided by the API architect. Architects will typically identify the broad structure of the API, which covers the resources, the associated HTTP methods (verbs) which act on the resources and the expected outcome (response) for each method. Designers will work with the architects and consumers to agree on the low level details such as the fields required for the request and response along with specific error scenarios which need to be supported.

API designers typically focus on questions like *"What does this service need to do?"* or *"What does this service need to provide?"*, while **API Con sumers** are focused on *"How can I use this API to do what I need?"*, or more accurately, *"How can I spend the bare minimum of effort to get what I need out of this API?"*.

API designers will typically focus on the following:

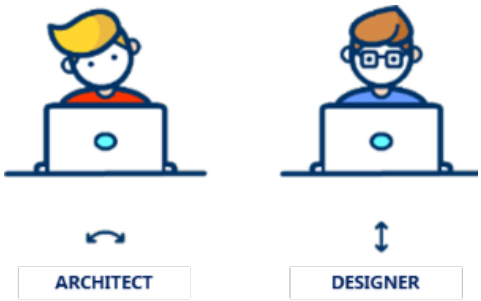
1. **Understanding the important details of the application** (High level Design Documentation) for which an API is to be created, so that an informed decision can be made as to what functionality needs to be exposed, how it needs to be exposed, and what functionality can instead be left out.
2. **Modelling this functionality** in an API that addresses all use cases that come up in the real world, following the RESTful principles as closely as possible.
3. **Best practices and patterns** for modern API design.
4. **Understanding the business vision**, value for a well-scoped business domain, and translating it to a great API Design.
5. **Improving Developer Experience (DX)** to ensure they can work quickly and more easily.

Deliverables

API Designers will deliver the following **artefacts**:

- **Swagger spec** describing the API interface, according to Westpac Mesh standards. Allows capability to generate code from Swagger and is based on the Golden Template.
- **Technical documentation** showing the internal design details required for the API based on Westpac Mesh Templates. This includes details such as: sequence diagrams, interface to provider mapping, and business logic.

Handover from Architect to Designer



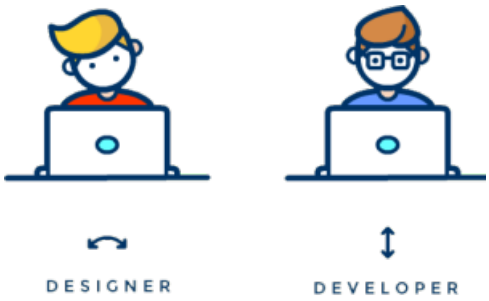
A meeting should be scheduled for a formal handover to be initiated. This meeting should provide a walk through by the architect of the high level use cases for the API for context what the API is intending to support and how it fits in the overall solution.

It is **your responsibility** to schedule a handover meeting with the architect.

At a minimum the following pieces of information for the API must be provided in the meeting before a design of an API can begin:

- Endorsed API names
- Endorsed Resources and operations of the API
- Business Owner, Domain Owner and Support Team details

Handover from Designer to Developer



It is **your responsibility** to schedule a meeting with the developer.

Designers to complete the following steps:

- Create a page for detailed design documentation under - Westpac Mesh API Design Register
- Get the specification reviewed by a Chapter Lead

2 REST and the Swagger Specification

Introduction to REST

What is REST (Representational State Transfer)?

REST (REpresentational State Transfer) is *an architectural style* for designing networked applications. The term was coined by Roy Fielding in 2000 in his now-famous PhD dissertation Architectural Styles and the Design of Network-based Software Architectures. The REST architectural style has been used to guide the design and development of the architecture for the modern Web and thus is based on HTTP concepts.

The 3 core concepts in REST are: resources, methods (verbs) and response codes.

What is a RESTful API?

Conforming to the REST constraints is generally referred to as being “RESTful”.

Complying with these constraints, and thus conforming to the REST architectural style enables any kind of distributed hypermedia system to have desirable emergent properties, such as performance, scalability, simplicity, modifiability, visibility, portability, and reliability.

The key components of REST are: resources, actions and response codes. Resources are used to encapsulate the information to be processed and are represented in the form of URIs (eg. http://api.example.com/customers). Actions are represented as HTTP methods (verbs) which include GET, PUT, POST, DELETE and are used to indicate the nature of processing required. Response codes indicate whether processing was successful or not; typical response codes are 200 (Ok), 400 (Bad Request), 404 (Not Found), 500 (Internal Server Error).

Key points for successfully offering a RESTful API are:

- Use HTTP Verbs as intended in RFC7231
- Provide sensible resource names
- Use HTTP response codes to indicate status
- Create resources at the right level of granularity

What is a Resource?

Resources form the nucleus of any REST API design. Any information that can be named can be a resource: a document or image, a temporal service (e.g. "today's weather in Los Angeles"), a collection of other resources, a non-virtual object (e.g. a person), and so on. It is very important to select the right resources and model the resources at the right granularity while designing the REST API so that the API consumers get the desired functionality from the API's.

The starting point in selection of resources is to analyse your business domain and extract the nouns that are relevant to your business needs. More importantly, focus should be given to the needs of API consumers and how to make the API relevant and useful from the perspective of API consumer interactions. Once the nouns (resources) have been identified, then the interactions with the API can be modelled as HTTP verbs against these nouns. When they don't map nicely, we could approximate. For example, we can easily use the “nouns in the domain” approach and identify low level resources such as Post, Tag, Comment, etc. in a blogging domain. Similarly, we can identify the nouns Customer, Address, Account, Teller, etc. as resources in a banking domain.

Further reading on what is a resource: https://www.thoughtworks.com/insights/blog/rest-api-design-resource-modeling

Introduction to OpenAPI

What is OpenAPI?

OpenAPI is a framework for describing your API using a common language that everyone can understand.

What is an OpenAPI Specification?

An OpenAPI Specification is a set of rules for a format describing REST APIs. The specification defines the interface of the API describing all supported resources, operations and expected responses in a format that makes it easier to develop and consume an API. It is an easy-to-learn, language agnostic, and both human and machine readable

Further reading: https://swagger.io/docs/specification/about/

Benefits of OpenAPI

- **It's comprehensible for developers and non-developers.** Product managers, partners, and even potential clients can have input into the design of your API, because they can see it clearly mapped out in this friendly UI.
- **It's human readable *and* machine readable.** This means that not only can this be shared with your team internally, but the same documentation can be used to automate API-dependent processes.
- **It's easily adjustable**. This makes it great for testing and debugging API problems.

OpenAPI (Swagger V3) Support

The Mesh Platform initially only had support for Swagger v2. This support has now been extended to OpenAPI (Swagger v3.0.3), and is the recommended contract specification for new Mesh APIs.

Existing APIs with a swagger specification will continue to be supported in the Mesh.

The Mesh Design guidelines apply to both OpenAPI and Swagger contracts.

The Golden Template

The golden swagger specifications are the template that all Westpac Mesh API specifications are to use when designing a Westpac Mesh API. **You must always go to the repository and get the latest golden swagger specifications whenever you are extending an API.** Newly created APIs will already be initialised with a sample swagger specification based on the latest golden template.

Whenever you extend an API you must ensure that any changes you make are backwards compatible. This includes any changes you make when incorporating changes in the latest golden specification.

For example:

- Changes to the information block are backwards compatible
- Adding a new mandatory header field is not backwards compatible

If changes cannot be made in a backwards compatible fashion then discuss this with your architect, as a new major version may be required. **Further reading** - API Design Standards#versioning

This repository contains the latest golden templates and some sample implementations. Please refer to the Readme.md for more information.

Contract repository: https://bitbucket.srv.westpac.com.au/projects/WDP2/repos/contracts/browse

Contract Guidelines

API Information

Basic information help others understand what functionality your API has. This information is displayed in Westpac Mesh Connect, so it is important in helping consumers discover your API.

```
---
swagger: '2.0'

##############################################################################
#                          API Information                                   #
##############################################################################
info:
  version: "1.0.0"
  title: The name of the API (eg. Golden Template API)
  description: A detailed description of the purpose and scope of the API
  x-goldenVersion: 3.3.0

##############################################################################
#                    Host, Base Path, Schemes and Content Types              #
##############################################################################
host: localhost:8080
basePath: /layer/domain/subdomain1/subdomain2/apiname/v1
schemes:
  - http
produces:
  - application/json
consumes:
  - application/json

##############################################################################
#                                    Security                                #
##############################################################################
securityDefinitions:
  # API Key based Authentication
  API Key:
    type: basic
    description: The API consumer is expected to present the API key and secret (password) to acc

security:
  - API Key: []
```

- Version: The version of your API. This follows semantic versioning, and must be updated whenever changes are made to the API.
- Title: The display name of your API. This is typically determined during naming governance, and should closely match your API's shortname. eg. An API with name 'cap-technolsvcdsgndvlpmt-applnmgt-*meshcmpntmgt*-v1' should have 'Mesh Component Management' as the title.

- Description: A high level business description of what your API does.
- X-goldenVersion: The version number of the golden template your API is based on. When possible, this should always be kept in sync with the latest Golden Template.
- Host/basepath/schemes: These values should already be auto-generated, and should not be modified.
- SecurityDefinitions: For non-proxy APIs, this should not be modified. APIs on the internal gateway will communicate with basic authorisation.

Parameters

Your swagger will come with predefined mandatory headers that are used internally by Westpac Mesh. You typically should not modify these parameters, but feel free to add your own query/path parameters as required.

Note: If using OpenAPI, these headers are automatically injected for you at runtime, and you do not need to manually declare them.

```
38 ##############################################################################
39 #                                  Parameters                                #
40 ##############################################################################
41 parameters:
42   x-messageId:
43     name: x-messageId
44     in: header
45     description: Must be a GUID (Globally unique message identifier). This uniquely identifies this call. This MUST NOT be propagated
46     type: string
47     required: true
48   x-appCorrelationId:
49     name: x-appCorrelationId
50     in: header
51     description: Must be a GUID (Globally unique message identifier). Consumer generated message identifier for correlation purposes. e.
52     type: string
53     required: true
54   x-organisationId:
55     name: x-organisationId
56     in: header
57     description: The organisation the caller is operating in.
58     type: string
59     required: true
60     enum:
61       - WPAC
62       - SGB
63       - BSA
64       - BOM
65       - BTPL
66       - UBS
67       - VIRG                #To be extended as required.
68       - RAMS
69       - BAS
70       - WNZL
71   x-channelType:
72     name: x-channelType
73     in: header
74     description: DEPRECATED. The type of channel from which the request has originated.
75     type: string
76     required: false
77     enum:
78       - Branch
79       - Online
```

- X-messageId: A GUID to represent a single transaction between a single component and another component on the platform. When implementing your API, this value should NOT be propogated.
- X-appCorrelationId: A GUID to represent an end-to-end transaction, across multiple components. This value should be propagated, Westpac Mesh providers helper libraries to assist with implementation.
- X-organisationId: The organisation the caller is operating in. Note this is not the same as brand.
- X-originatingSystemId: The Alfabet Id of the calling application
- X-consumerType: The originating consumption channel invoking the API.
- X-userId/X-userIdScheme: These should be used to identify the user invoking the API if required.

Paths

Here you can define your resources/endpoints required by your API. Note that the GET /healthcheck is mandatory and should not be modified.

```yaml
##############################################################################
#                      Paths                                                 #
##############################################################################
paths:
  /sampleResources:
    get:
      tags:
        - getSampleResources
      description: Retrieve list of sampleResource objects
      operationId: getSampleResources
      parameters:
        - $ref: '#/parameters/x-messageId'
        - $ref: '#/parameters/x-appCorrelationId'
        - $ref: '#/parameters/x-organisationId'
        - $ref: '#/parameters/x-channelType'
        - $ref: '#/parameters/x-originatingSystemId'
        - $ref: '#/parameters/x-consumerType'
        - $ref: '#/parameters/brandSilo'
        - name: customerRating
          in: query
          description: customerRating assigned to the sampleResource
          type: string
        - name: pageSize
          in: query
          description: the maximum number of items to be returned in this call
          type: number
        - name: pageKey
          in: query
          description: The key identifying the page of data to be returned
```

- Tags: This will appear as a title for your endpoint when viewed in the Swagger Editor.
- OperationId: This will determine the name of the related classes when you generate code from your swagger.
- Parameters: Mandatory headers will already be defined here. Feel free to additional parameters more as required.
- Responses: Here you can reference any defined response objects for each HTTP error code.

Responses

Here you can define your response objects. Please note that HTTP 200 responses should ALWAYS have a 'data' object at the root level, which wraps your response entity. At the root level, you may also have a 'paging' object.

```yaml
##############################################################################
#                              Responses                                     #
##############################################################################
responses:
  #Success Responses
  200GetSampleResourceListResponse:
    description: OK
    schema:
      properties:
        data:
          type: object
          properties:
            sampleResources:
              type: array
              items:
                $ref: '#/definitions/SampleResourceList'
        paging:
          $ref: '#/definitions/Paging'
```

HTTP 400 and 500 error responses should list out all possible error codes and messages in their description. A table format should be used for ease of reading.

```
# Client Error Responses
400GetSampleResourceListErrorResponse:
  description: |
    __Bad Request__

    Possible errors

      | Error Code | Error Message | Error Description |
      | ------- | -------- | --- |
      | n   | A concise description of the error  | Detailed information about the error.  Be careful when exposing provider errors that sensitive information is not
      | 1   | Invalid brandSilo parameter | The required brandSilo parameter is missing or invalid |
  schema:
    type: object
    properties:
      result:
        type: object
        properties:
          errors:
            type: array
            items:
              $ref: '#/definitions/Error'
```

Definitions

Here you can define your response objects. You should write object definitions with the intent of reuse across your swagger. Some other points to consider:

- Always include examples where possible. Ideally, examples should be at the field level, rather than the object level.
- Pay attention to the naming of your definitions, as these your generated classed will have the same names code is generating from the swagger.
- Enums should not contain spaces, and have consistent casing

Further Examples

You can find more examples in the Contracts repository and in Mesh Connect, by searching for 'Reference Implementation'.

Sample OpenAPI contracts:

https://mesh.westpac.com.au/components/apis/reference-scoped-access-v1?tab=contract

https://mesh.westpac.com.au/components/apis/mesh-component-quality-metrics-v2?tab=contract

Sample Swagger contracts:

https://mesh.westpac.com.au/components/apis/inf-bnkng-resrcitm-logclitcmpnt-meshcmpnt-v1?tab=contract

Proxy API Guidelines

Proxies are components that have an interface like an API, but do not contain any code implementation. They are used as passthroughs to actual APIs that are on a separate host. All outbound access to vendor service/ API or an inbound access by a partner is always through a proxy API that is hosted in Westpac Mesh API gateway .

Vendor must supply a service specification/ contract in the form of Swagger / Open API definition (for Restful API's) or WSDL (for SOAP based Web services). The Proxy component also should have environment configuration and security policies as per the Supported Authentication

Different Golden Templates are used for the different API Proxy use cases.

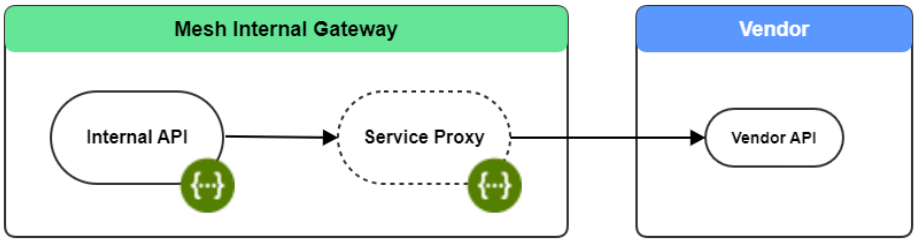
Service Proxy: Outbound access to Vendor API

For this use case where you need to consume either an external vendor API, or a non-Westpac Mesh internal API , you require a Service Proxy. Eg: MS APIs / IBM Watson APIs etc

For a Service Proxy, you can use the Golden Template for a standard API, with the following considerations:

- As the resource definitions will follow what is specified in the vendor's API, the Proxy swagger does not need to strictly follow Westpac Mesh design guidelines. However some effort should be made to sanitise the swagger so it is readable and reusable for a broader audience.

- x-headers which are mandatory for APIs running on Westpac Mesh can be removed. They should not be sent to external vendors. X-headers may be sent to non-Westpac Mesh internal APIs for logging/audit purposes.
- 'Vendor name' is suffixed to the title of the API to indicate it is a vendor Proxy



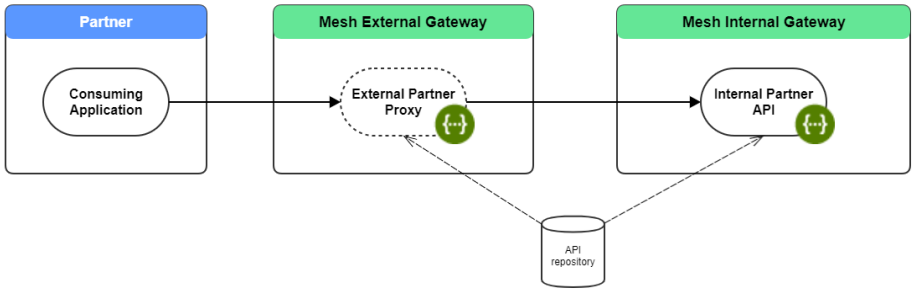
Further reading : Pattern 3: Vendor Supplied API

Partner Proxy: Inbound access to Mesh API

To expose our products and services to partners that have an established relationship with the group we use Partner Proxies / APIs . This is hosted on the External Partner Gateway. Eg: Parsifal / Open banking

Partner APIs require two Swagger specifcations generally. One for the external gateway which our partners will see and one for the internal gateway which will reflect the additional headers injected by the external gateway. With the Simplified Partner Proxy pattern, you only need a single repository with one Swagger specification. The additional configuration required for the external proxy will be created automatically.

The external GW interface is just a proxy and the internal partner is a Java API.



Further reading: Simplify Partner Proxy

The Partner Proxy swagger should be analogous to that of the internal API. The key difference that the Partner Proxy swagger will be based on the Partner Proxy Golden Template.

- SecurityDefinitions will need be set for the External Partner Proxy.
- Certain mandatory headers such as x-originatingSystemId and x-consumerType, are injected by the gateway, and can be removed from the External Partner Proxy's swagger. Tracing headers x-messageId and x-appCorrelationId can be passed through by the Partner if required.
- If the interface is driven by Westpac , then the Proxy will also adhere adhere to Westpac Mesh guidelines.
- If the interface is provided by the vendor / follow an industry standard such as open banking then it will conform to their standards
- The External Partner Proxy will be defined as agreed upon by the Partner and development team. This Proxy does not necessarily need to follow Westpac Mesh guidelines, but should still be based on the Partner Proxy Golden Template.
- The Internal Partner Proxy will mirror the swagger of the External Partner Proxy, but will contain code for transforming the incoming request to match the specifications of the internal API.

Configuring your Proxy

You can read more about how to configure your Proxy API at the below link.

Further reading:

Proxy use case scenarios

API Naming Standards#Exposinginternalapplicationout-of-the-boxAPIsorWSDLstoPartners

Exercise: Getting started editing a Swagger specification

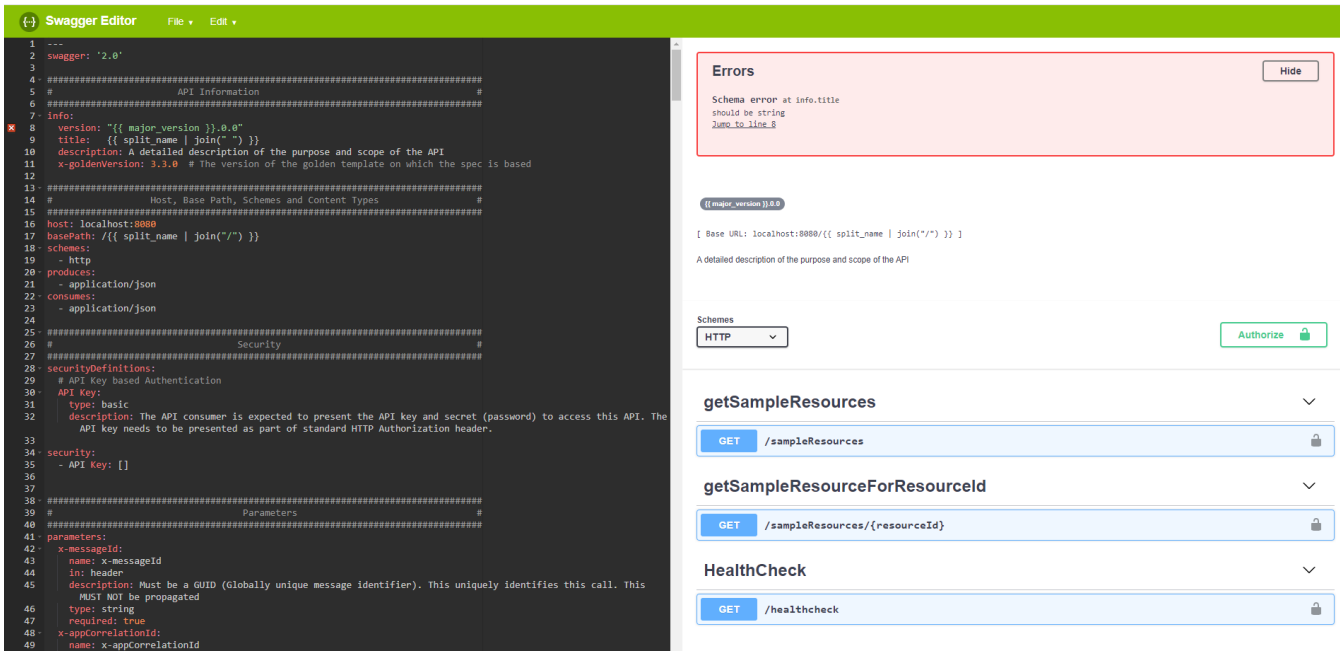
As a Designer, you should be comfortable editing a Swagger specification. Note that while the Golden Template is typically auto-populated for you when creating a project, for the purposes of this exercise, we will attempt to fill in the Swagger ourselves.

First, let's open the Swagger Editor.

Note: Westpac has hosted Swagger Editor internally. The link above is the internal one. DO NOT use the one available in internet

In another window, find the Swagger Golden Template in the contracts repository. The Readme.md should give you some indication on where to find this if you are stuck.

Open the file, copy and paste the contents into the Swagger Editor. You should see an error as below:



We are seeing errors, because the template has not been populated yet. We want to replace the braces {{ }} with some meaningful data.

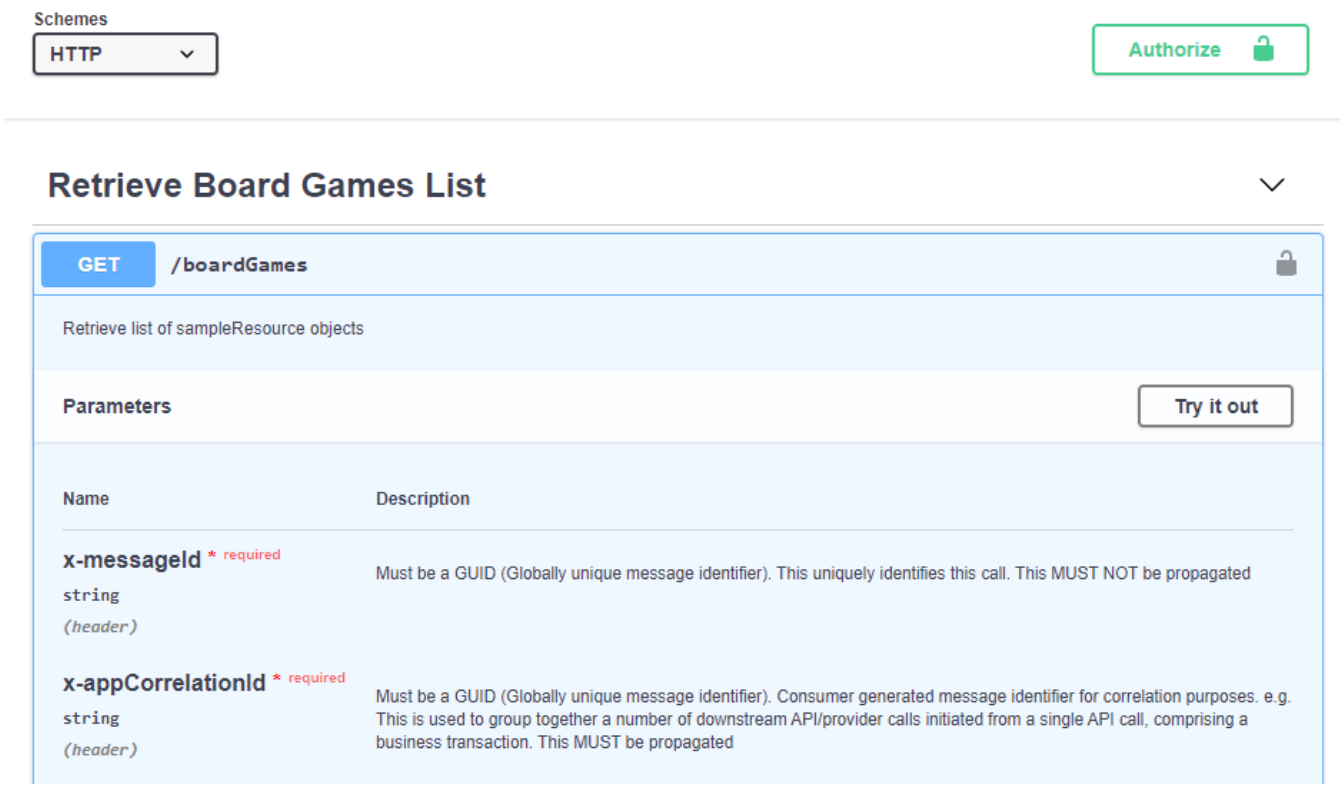
- Update the version to: 1.0.0
- Update the title to: Board Game Retrieval
- Update the description to: This API retrieves a list of board games and their details.

This should get rid of the error. Now let's do some basic editing.

Scroll down to the first resource.

- Update /sampleResources to /boardGames
- Update the tag to: Retrieve Board Games List
- Remove the brandSilo parameter

The right hand side of the screen should update with your new details to look something like below:



Now let's edit the response object. Note the the response objects are all references as denoted by **$ref**.

Let's find the response object for a HTTP 200. Hit Ctrl+F and Search for 200GetSampleResourceListResponse.

```
228     responses:
229       #Success
230       '200':
231         $ref: '#/responses/200GetSampleResourceListResponse'
232       # Client Errors
233       '400':
234         $ref: '#/responses/400GetSampleResourceListErrorResponse'
235       # Server Errors
236       '500':
237         $ref: '#/responses/500ErrorResponse'
238
```

Notice how the schema of 200GetSampleResourceListResponse contains two attributes:

- data: this is object is mandatory, and expected for all HTTP 200 responses. You can define the contents of your response object in here.
- paging: an optional object we can use to describe the subset of data that we are viewing when we have a large result set. We will learn more about paging later.

Let's make some updates to this object.

- Update the name of the array to boardGames
- Delete the example block that you find under the 200GetSampleResourceListResponse object. It is easier to maintain your examples by setting them at the field level.

Again, notice how the right side has updated (you might need to scroll down to find the response example).

Code	Description
200	*OK*

Example Value | Model

```
{
  "data": {
    "boardGames": [
      {
        "id": "string",
        "name": "string",
        "customerRating": "superb"
      }
    ]
  },
  "paging": {
    "count": 0,
    "pageSize": 0,
    "nextPageKey": "string",
    "previousPageKey": "string"
  }
}
```

The elements of the array are defined by SampleResourceList. Search for this object and edit the fields with some examples.

- Update id with an example. Underneath the description line, add: example: "123". Note that quotes are required, as the example could be incorrectly interpreted as a numeric.
- Update name with an example.

You should see something like below:

Code	Description
200	*OK*

Example Value | Model

```
{
  "data": {
    "boardGames": [
      {
        "id": "123",
        "name": "Race for the Galaxy",
        "customerRating": "superb"
      }
    ]
  },
  "paging": {
    "count": 0,
    "pageSize": 0,
    "nextPageKey": "string",
    "previousPageKey": "string"
  }
}
```

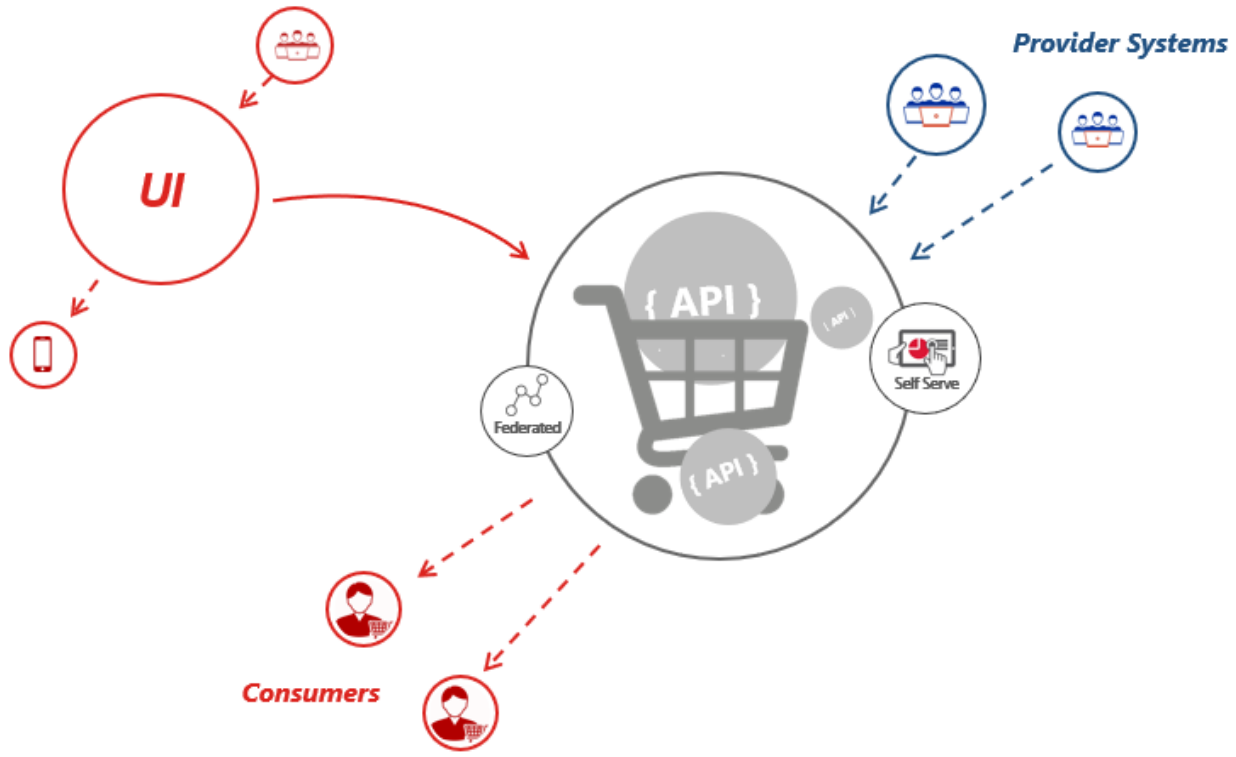
Finally, using what you have learned, see if you can update the 400 error codes to look something like the below:

400

```
Bad Request
Possible errors

Error Code Error Message                    Error Description
---------------------------------------------------------------
100        Invalid customerRating query     The customerRating is
           parameter                        invalid

101        Invalid pageSize query           The pageSize is too
           parameter                        large

102        Invalid pageKey query            The pageKey is invalid
           parameter
```

3 Designer Tools

Mesh Connect for Designers



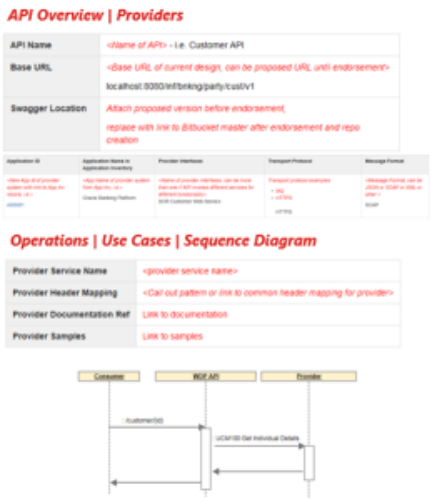
Multiple teams are producing and consuming APIs and other Mesh components, which are brought together in Mesh Connect, our marketplace.

Designers can use Connect to explore APIs and their swagger specifications on the Mesh.

Some of the features useful for Designers include:

- Search for a Component by name, type, tags, description
- View Component details, including description, ownership and support details
- Links to the detailed technical design documentation and swagger specifications for a Component
- View the consumption and dependency model for a Component
- View the ownership model by Domain

API Design Template

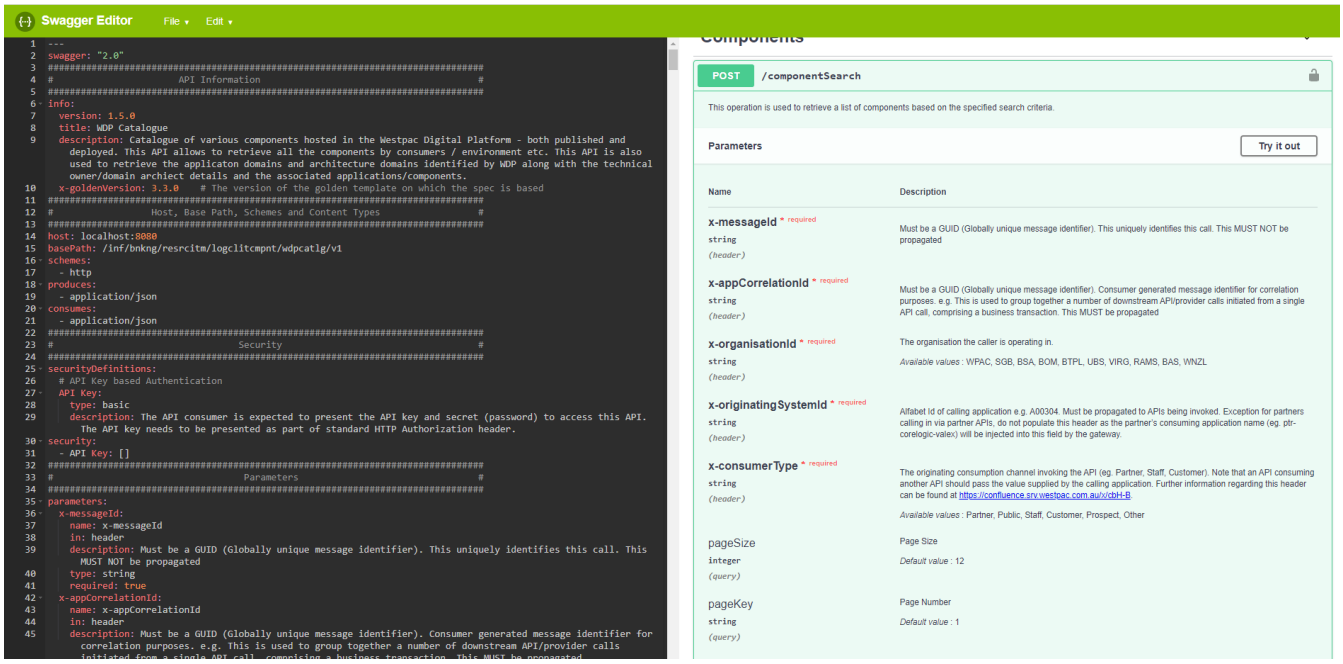


The API Design Document aims to bridge the information gap between swagger and provider interactions and is used by developers, testers and operational personal.

This API Design Template provides an easy starting point for designers. Further instructions can be found in the template.

New API Design Documents can be added to the Mesh API Design Register. You can also find many examples for existing APIs here.

Swagger Editor



The Swagger Editor provides a user-friendly way to edit and debug your swagger specifications.

During Swagger Governance meetings, this is the preferred way to view and assess swagger specifications.

Mesh Features for OpenAPI

OpenAPI YAMLs

There are three different YAML files you will be working with in OpenAPI.

- `openapi-base.yaml` - This file contains the definitions for the mandatory platform headers. These headers will be automatically injected for you when used by the platform, and you do not need to manually include them. If you need to reference any of these definitions, you can import them with external references.
- `openapi.yaml` - This is the main contract file you will be working with, the resources and entities for your API can be defined here. When sharing the API contract with consumers, and generating code for development, you should be using openapi.yaml.
- `openapi-client.yaml` - This file will be automatically generated for you based on your openapi.yaml, and has the mandatory platform headers injected for you. You should use openapi-client when testing your API.

Detailed Instructions: How to Use OpenAPI 3.0 - Development guide

Reference Implementation: Reference API

Contract Linting

The Mesh provides tools to verify that your API contract meets the design standards and passes the mandatory checks.

This contract linting is run automatically when the API is built on the continuous integration pipeline, during digital signing and can also be executed manually using Mesh Developer Tools.

Note that the contract linting is currently only applied for OpenAPI, and not Swagger 2.0 contracts.

```
-----------------------------------------------------------------------------
Rule: Components Tracker validation rule
SUCCESS:  The component is correctly registered with .id value 104o
-----------------------------------------------------------------------------
Rule: Spectral Rule Execution
ERROR:    78:28    Request/Response content type should not be 'application/xml'. Actual: application/xml    paths./consumingApplicationTypes.get.responses.200.content.application/xml
ERROR:    171:14    202 response is only allowed for DELETE, POST and PUT operations    paths./userCommunities.get.responses.202
ERROR:    190:39    Content type application/merge-patch+json is only allowed for PATCH operations    paths./pets.post.requestBody.content.application/merge-patch+json
ERROR:    741:22    "emailAddress" property must match pattern "email"    components.schemas.domainContact.example.emailAddress
WARNING:  37:16    Parameter x-messageId is reserved for platform use and should not be included in OpenAPI spec    paths./layers.get.parameters.0.name
WARNING:  62:23    This content type is not recommended, and requires approval from Mesh governance    paths./abbreviations.get.responses.200.content.audio/mpeg
WARNING:  78:28    This content type is not recommended, and requires approval from Mesh governance    paths./consumingApplicationTypes.get.responses.200.content.application/xml
WARNING:  214:10    Operation tags must be defined in global tags.    paths./healthcheck.get.tags.0
WARNING:  235:21    This http code  is not supported    components.responses.400ErrorResponse
WARNING:  245:30    This content type is not recommended, and requires approval from Mesh governance    components.responses.400ErrorResponse.content.multipart/alternative
WARNING:  262:19    This content type is not recommended, and requires approval from Mesh governance    components.responses.500ErrorResponse.content.text/plain
WARNING:  266:38    An anonymous object  is  defined    components.responses.204UpdateConsumptionGrantResponse
WARNING:  272:19    This content type is not recommended, and requires approval from Mesh governance    components.responses.200GetComponentListResponse.content.video/webm
WARNING:  340:14    Fields should not be defined with a key-value pair    components.schemas.abbreviation.properties.value
WARNING:  698:30    Potentially unused component has been detected.    components.schemas.applicationDomainResponse
WARNING:  743:31    Potentially unused component has been detected.    components.schemas.architectureDomainResponse
-----------------------------------------------------------------------------
```

Detailed Instructions: How to Validate a Contract Using Spectral

List of design standards rules covered by automated linting: Contract Linting Rules

Resource Level Access Control

Consumption of your API can be controlled, so that consumers only have access to designated resources, instead of access to the entire API. To use this feature:

- Your API must be created with Scoped Access enabled
- Scopes should be defined in your openapi.yaml
- Resources should declare the eligible scopes in your openapi.yaml
- When a consumer requests access to your API, they declare which scope they will be using

Detailed Instructions: API Resource Level Access Control

Reference Implementation: Reference API

OpenAPI Specific Features

DRAFT

- Overview
- Changes

Overview

Mesh Platform support OpenAPI 3.0 along with the legacy Swagger 2.0. This page a walkthrough in terms of the design of OpenAPI 3.x specification, in comparison with Swagger 2.0

Changes

S. NO.	SWAGGER 2.0	OPEN API 3.0
1	All the swagger specifications should follow the Golden Template and its version is validated WDP2 / contracts / **templates / swagger.yaml** **[11:11]** master `x-goldenVersion: 3.5.0 # The version of the golden template on which the spec is based`	All the openapi specification should refer to the published version of golden template. As openapi golden template is a published artifact, any existing openapi specification can still refer to an old published version. ⚠ Schema version indicated in the master branch can be considered as a published version however, we have a release process to get this schema version available across all non-prod and prod environments. If you find any issues then, please reach out to Mesh Support Team Published schema version can be referred to **component** > **contracts** > **schema_version** in wdp-config.yaml
2	All the headers indicated in the golden template should be available in every component's swagger and these headers should be passed for every endpoint	All the headers available in golden template are platform specific and no longer required for the developer to write a code to manage these platform headers. Platform headers are available in https://schema.mesh.westpac.com.au/platform/contracts/0.12.0/openapi-base.yaml
3	HealthCheck endpoint available in the golden template should be available in every component's swagger and its reference model objects	Healthcheck endpoint should be available in the openapi specification however, the model objects for the healthcheck are available in openapi-base.yaml and is managed by the platform. What this means is, there is no additional check required for chapter leads or the developers to validate, if the endpoint and its model objects are intact
4	Basic Auth and Scoped Access are supported however, there is no approved /signed swagger with scoped access available currently on the Mesh Platform	Scoped Access and Basic Auth are supported. The common configuration or objects are referred from openapi-base.yaml **Example Template** - openapi (5).yaml

5	The common objects i.e., platform specific fields are copied in every specification and is referred like below:
	
	`schema:`
	`  $ref: '#/definitions/ErrorResponse'`
		Common Objects i.e., platform specific fields are not copied in every specification. Instead it is referred from openapi-base.yaml
		
		`schema:`
		`  $ref: 'https://schema.mesh.westpac.com.au/platform/contracts/0.12.0/openapi-base.yaml#/components/schemas/ErrorResponse'`

6	Only URL versioning was supported in swagger	Media type versioning is available for Open API 3.0 and is a flexible solution for versioning endpoints without creating a new major version of an API. patch: description: Retrieve a specific sampleResource object operationId: patchSpecifiedSampleResource security: - ScopedAccess: [writer] requestBody: description: Partially Update the resource required: true content: application/merge-patch+json: schema: $ref: "#/components/schemas/PartialUpdateSampleResource" **application/merge-patch+json; version =2.0:** schema: $ref: "#/components/schemas/PartialUpdateSampleResource_v2" **Example Template** - openapi (5).yaml
7	Swagger supports only **`allOf`** derivative to re-use the common model objects	OpenAPI support **`oneOf`**, **`anyOf, not`** and **`allOf`** derivatives that gives a much more flexibility for the contract designers to define their required and optional objects without any duplicates https://swagger.io/docs/specification/data-models/

	https://swagger.io/specification/v2/#schemaObject	

Exercise: Find an API in Mesh Connect

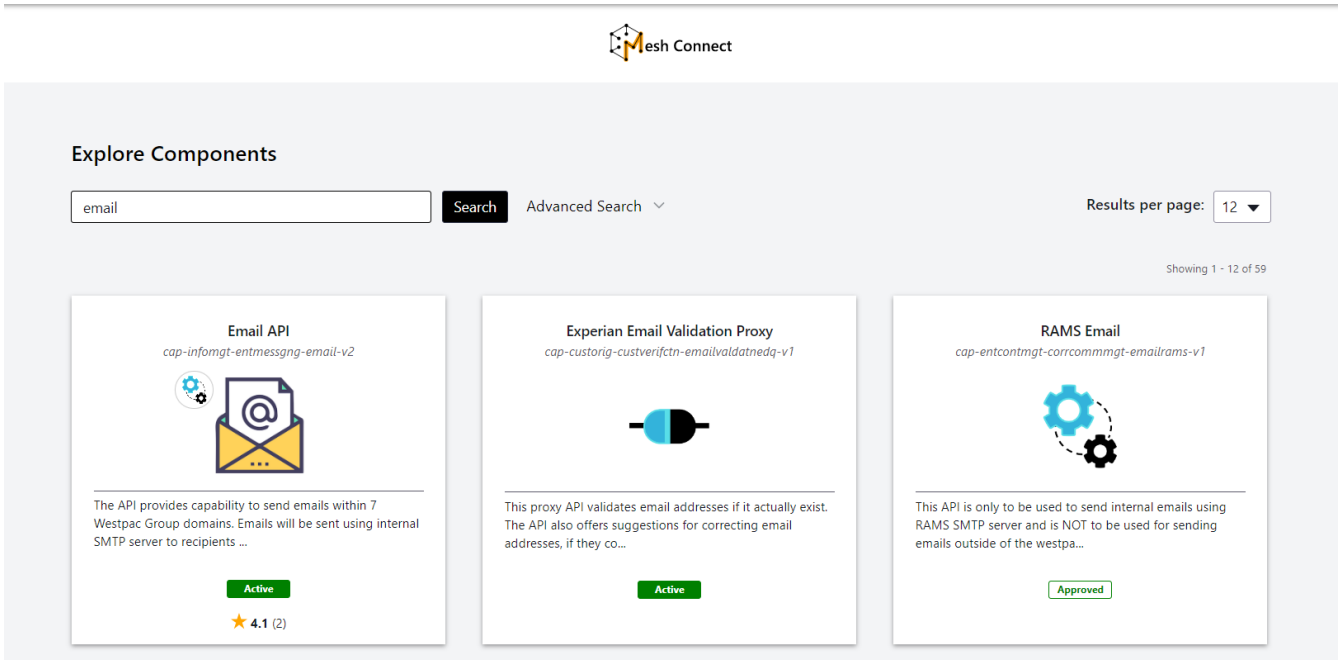
The following exercise attempts to get you familiar with some of the designer tools we have looked at. Please attempt the exercise on your own, before following the solution.

Your project is looking for a Capability API to consume, that is able to send emails with an attachment to Westpac Group domains. You will need to find such an API, and view to specification to see if it is the right fit for your project. Additionally, you will also want to contact the owners of API to potentially raise a consumption request.

1. Using Mesh Connect, find an API that fulfils your business requirements
2. Open the specification in the Swagger Editor. Find out what file types are currently supported for attachments.
3. Find out who the Technical Delegates are for this API.

Mesh Connect Search

Load up Mesh Connect in your browser and browse to the API search (you can also search all components from the home page). In the search field, enter some relevant search text, such as 'email' or 'attachments'.



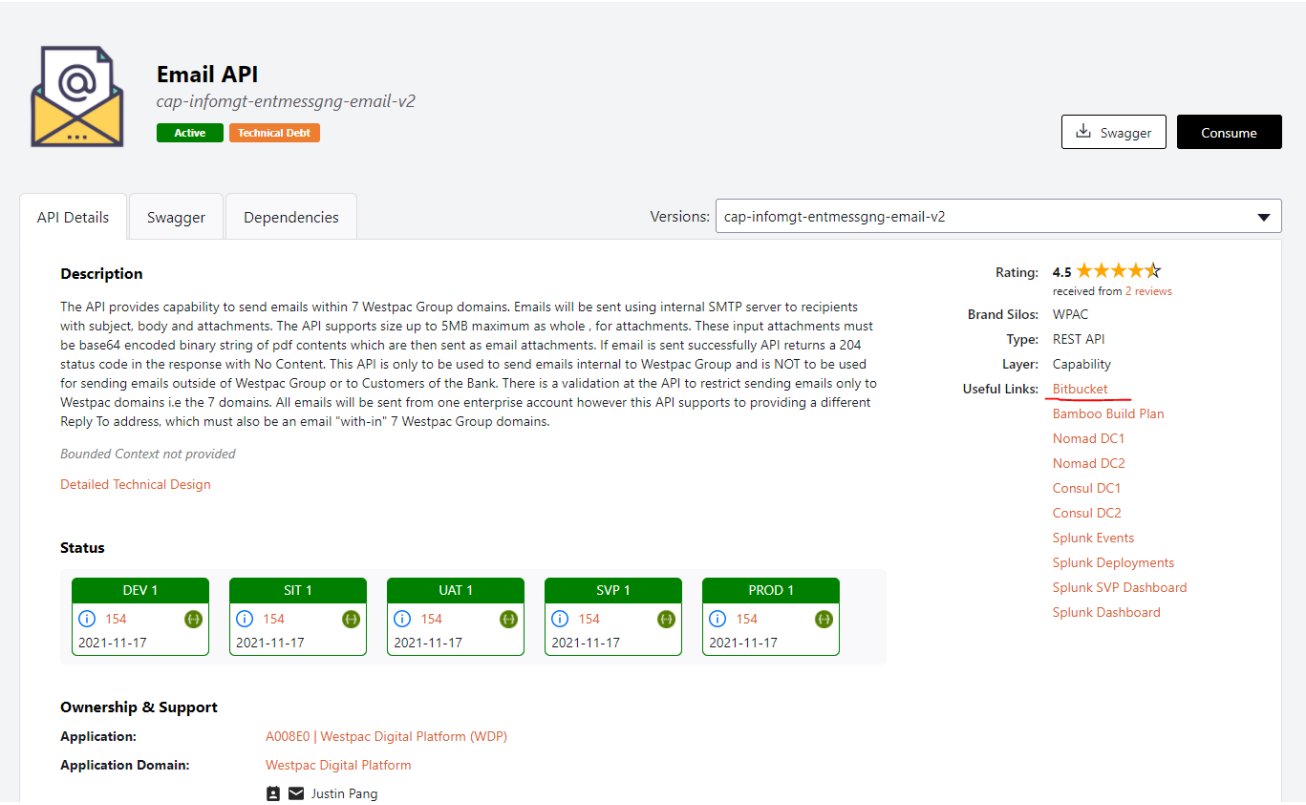
Once you execute the search, one of the APIs that should be returned in the results is 'Email API'. The description summary should give us some indication this API will suit our purpose. Click into this to link to get the component details.

Component Details Page

On this page you should be able to get a full description of the API, deployment status, a summary of the resources available, and some other useful links.

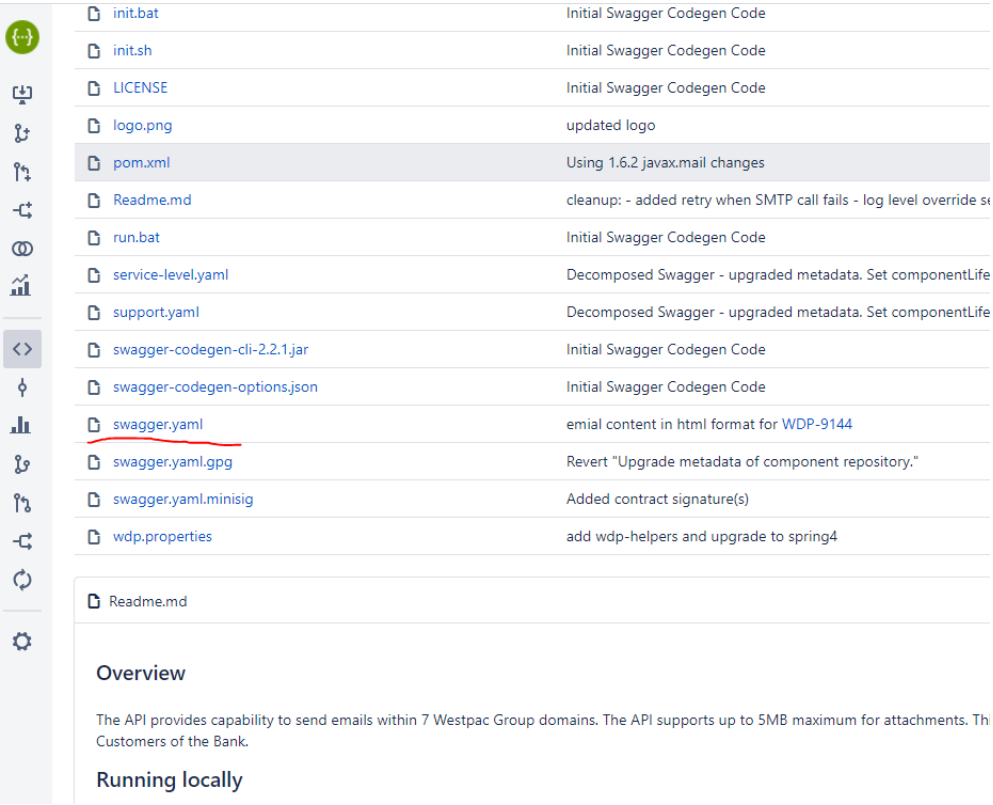
We should actually be able to get the information we need by clicking on the 'Swagger' tab, but lets dig a little deeper.

On the right side of the details page, find the Bitbucket link under the 'Useful Links' section.



Clicking this link will bring us to the Bitbucket repository of this project where can find all the code for this API.

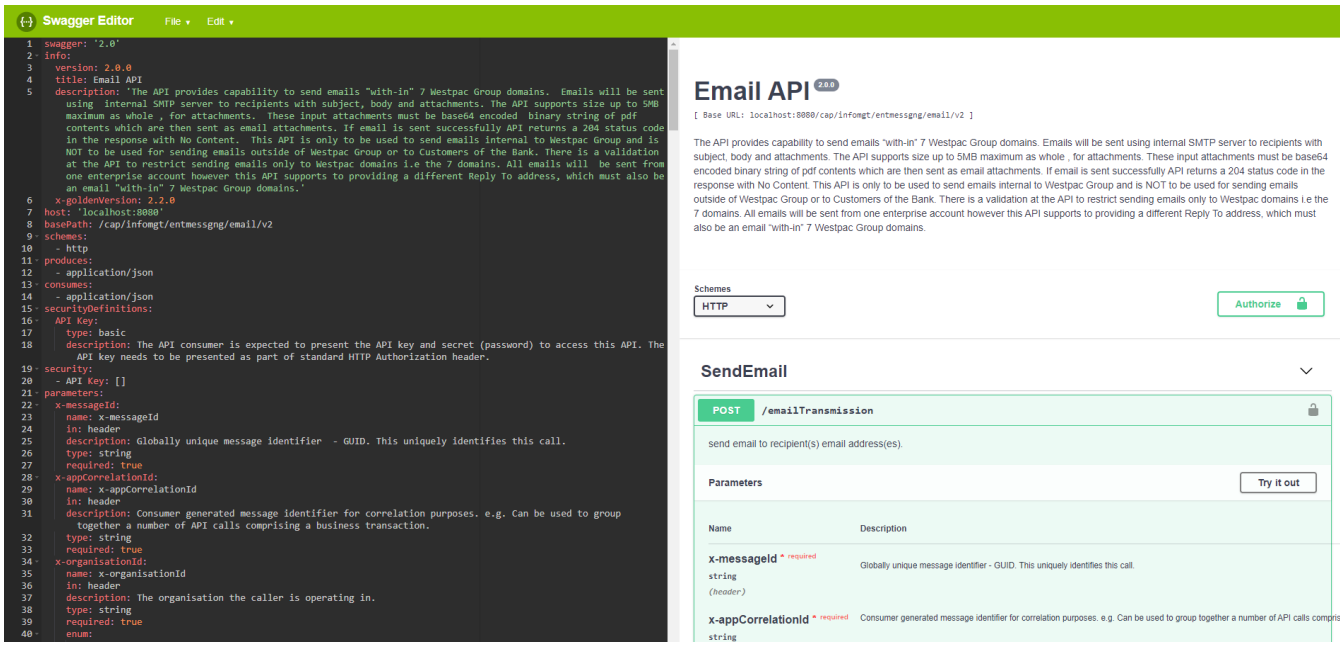
Scroll down until you find swagger.yaml. Open the file, and copy the contents.



Using the Swagger Editor

Now open the Swagger Editor, and paste the contents of the swagger.yaml you copied, into the left side of the editor.

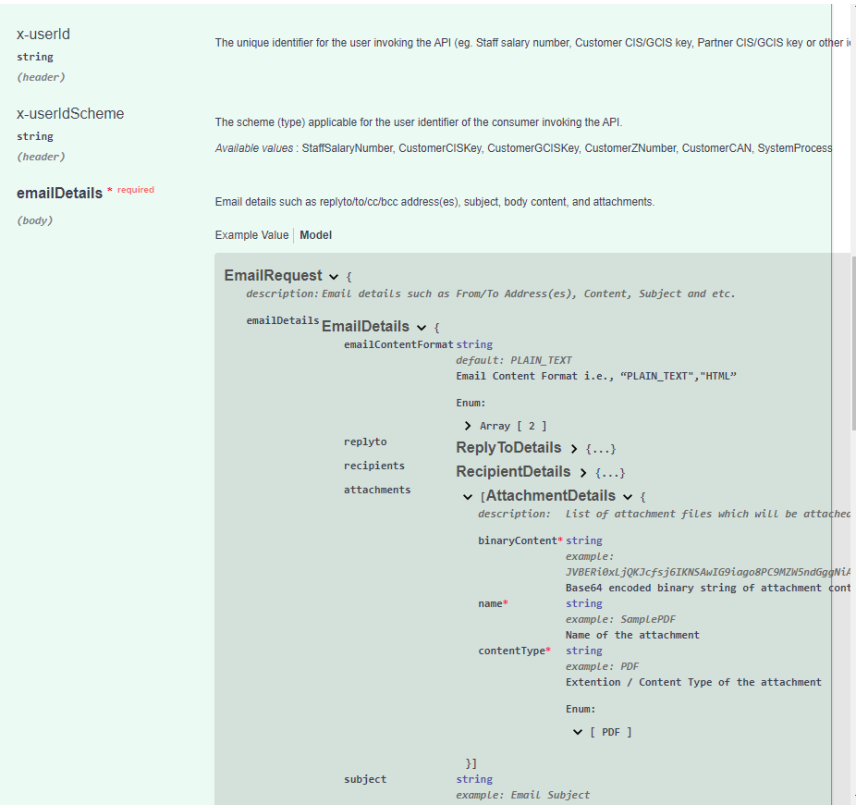
You will notice the right side of the editor updates to display the API specification, with the list of resources summarised.



Click on '/emailTransmission' to view the full details of this resource. You should be able to view the required input parameters, and sample request payload, and response codes as you scroll down.

Click on 'Model' next to the request body definition (EmailDetails). The description of each individual field in the request body are described here.

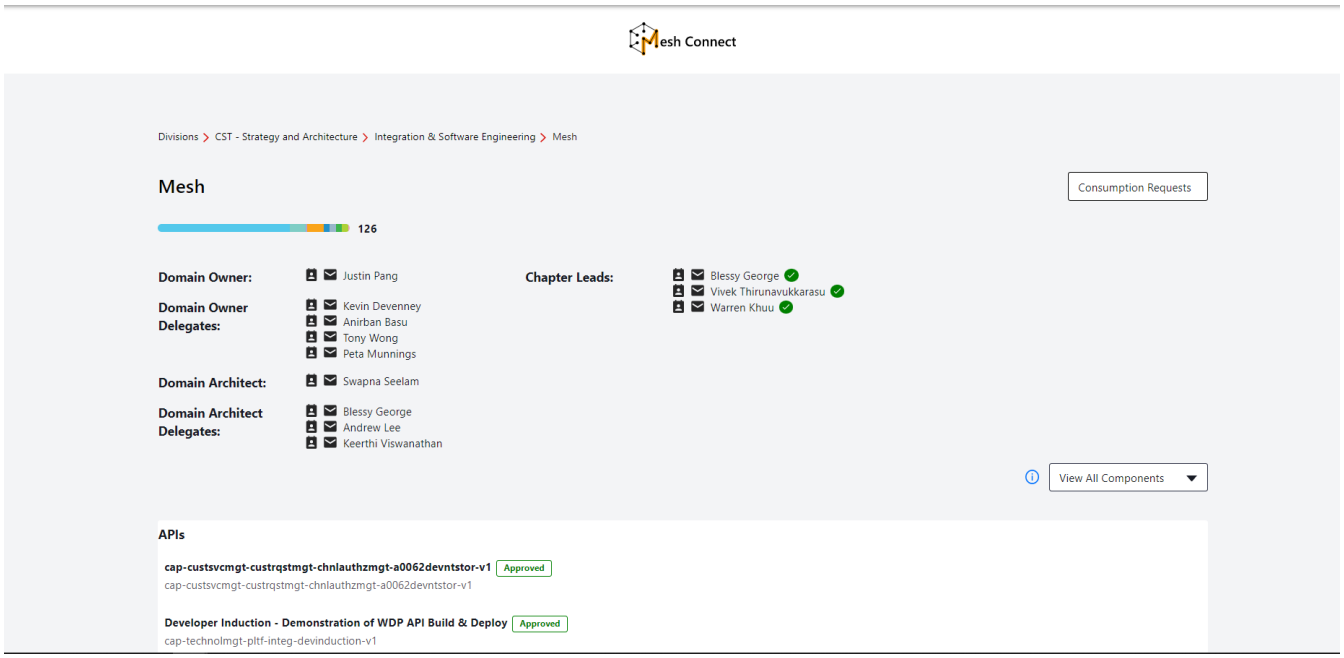
Drill down further until we find the allowed content type for attachments. We should see that the only type supported currently is PDF.



Viewing Ownership Details

Going back to our component details page in Connect, if we scroll down further, we should be able to find Ownership and Support details for the component. The details here are sourced from the Alfabet system, and from models maintained by the Mesh team.

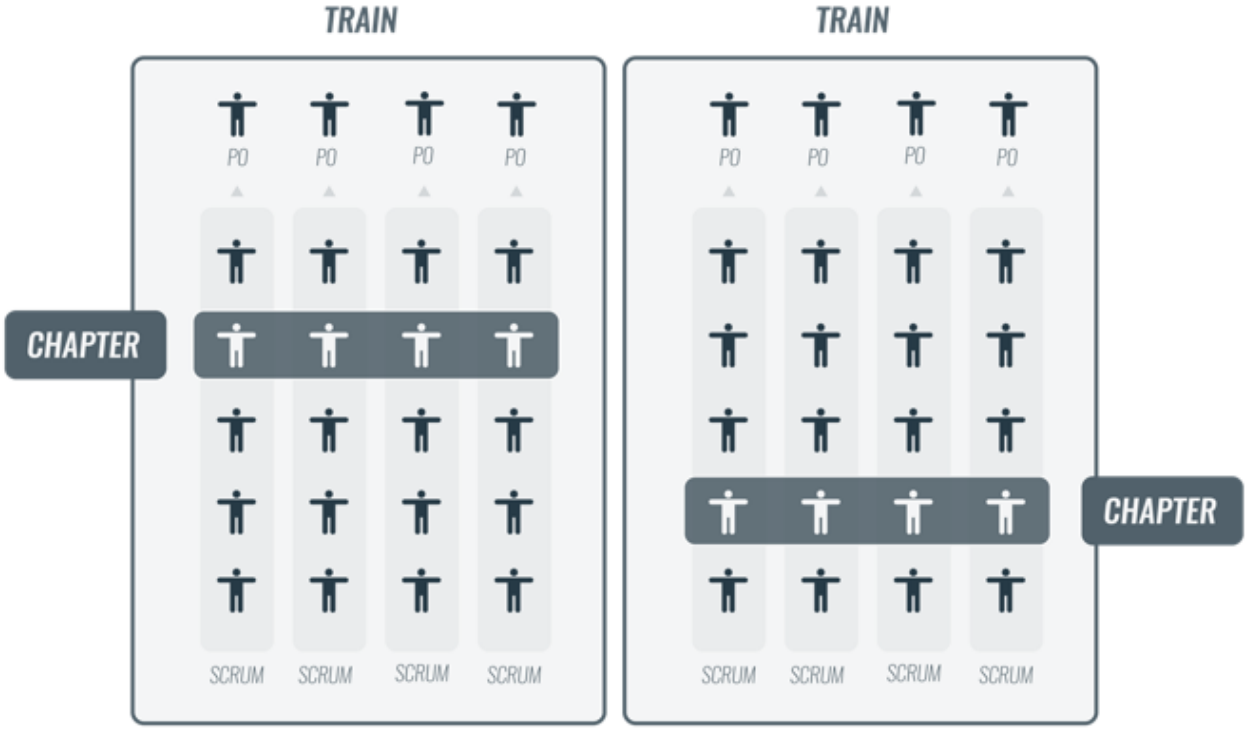
The Domain summary only displays the domain name, and Domain Owner. Let's click the domain name to find out more details.



This takes us to the Domain details page for Westpac Digital Platform. Here we can see the full details of the domain, such as owned Applications and their associated components, Chapter Leads, and the Domain Owner Delegates that we are looking for.

4 API Contract Governance

Chapters



The chapter is a small group of people having similar skills and working within the same general competency area, within the same domain. Each Chapter has a chapter lead who is responsible for enabling the team members to perform their role and is accountable for the quality

Chapter Leads

Chapter leads are responsible for:

- Quality of the Swagger Specification.
- Well documented API Design Documentations.
- Driving API endorsement by helping with pre-reviews and presenting at the swagger governance forum.
- Review the Detailed Assessment of the domain score card and take the actions to fix identified issues at a component level to ensure that the platform compliance criteria is met.
- Ensure your components are appearing in Westpac Mesh Connect with proper title, description, status etc.
- Deprecate and retire components which are no longer needed.

API Contract Approvals

An chapter lead is given the authority to approve API Contracts for their Mesh Domain without requiring explicit governance approval, however they will be held accountable for the quality of these deliverables.

Further reading: Chapters and Guilds

Contract Approvals

API contracts must be reviewed and digitally signed by a Chapter Lead before they can be deployed to UAT, SVP and Production environments.

Status	Approved
Approved by	Warren Khuu

Intended Audience for this Guide:

Developers / Designers / Chapter Leads.

Use Case:

To deploy an API in UAT, SVP and Production environments, the API contract must have an attestation from the domain's chapter lead.

This feature allows domains to manage API contract attestations through Mesh Connect.

Both Swaggers and OpenAPIs are supported through this process.

Before you Start:

Your domain should have a Chapter Lead, who will provide the attestation. You can see who your Chapter Leads are on the details page of your Mesh Domain, Endorsed Chapter Leads are indicated with a badge.

Find out more about nominating chapter leads and the endorsement criteria here: Westpac Mesh Chapter Lead

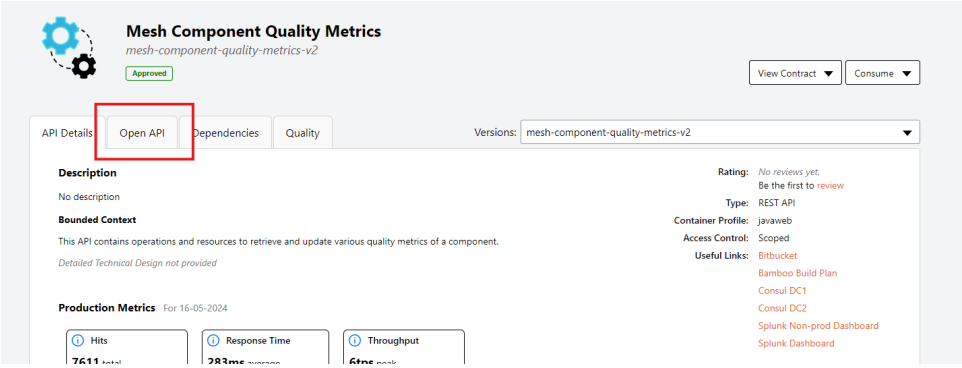
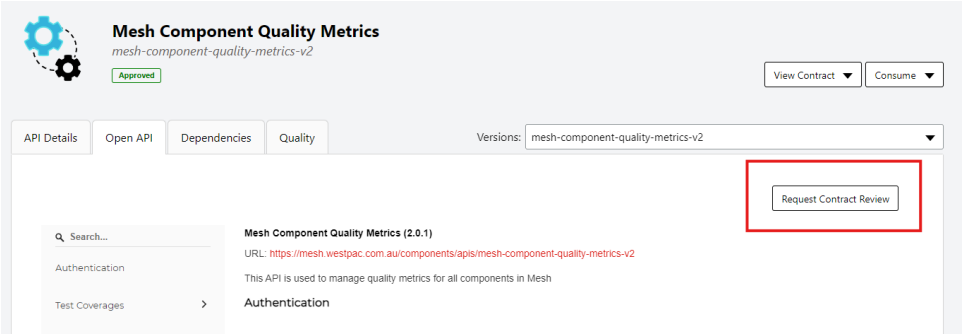
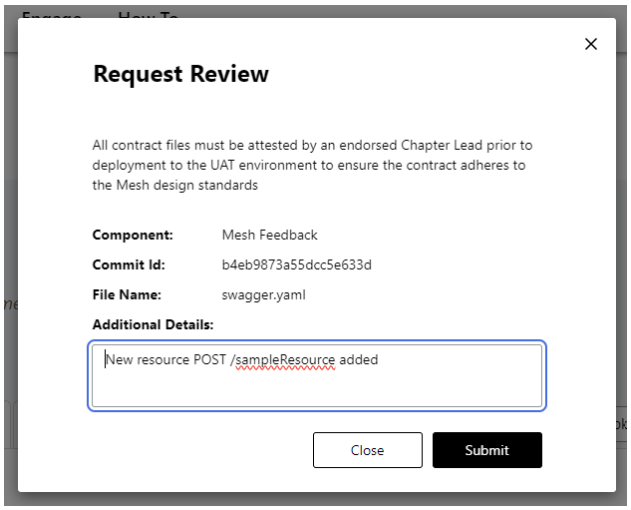
In the case your domain does not have a Chapter Lead, the requests will be sent to the Mesh Central Governance team, who will provide the attestation on your domain's behalf.

Any changes to your Swagger or OpenAPI have been merged to the master branch in your repository.

> ⚠ This attestation process intended to replace the existing digital signing process for API contracts.
>
> This new process will NOT create a .minisig file in your repository, as it is no longer required if attesting via Mesh Connect.
>
> While the digital signing process continues to be supported, we plan to deprecate this in future.

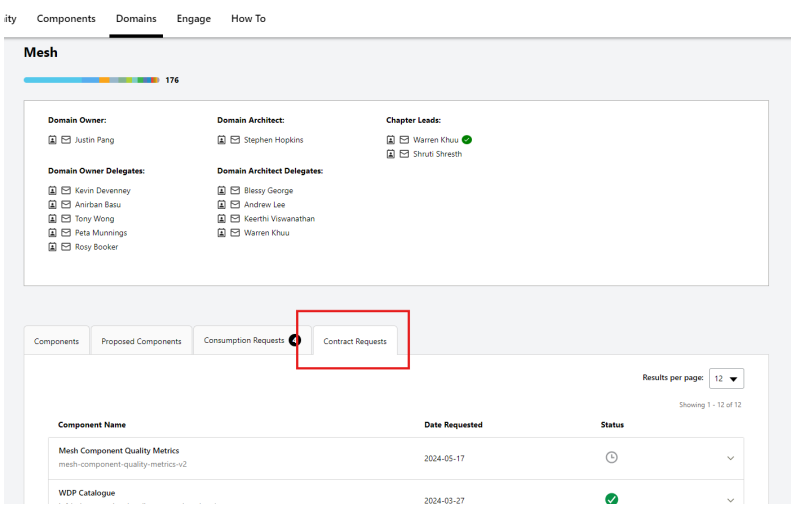
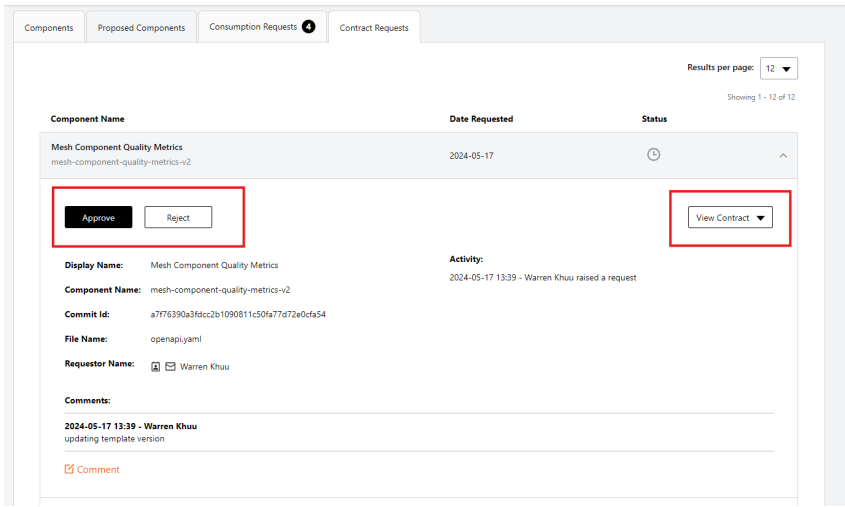
Steps:

Requesting API Contract Attestation

#	Action	Comment
1.	Navigate to your component details page in Mesh Connect. Select the Swagger /OpenAPI tab.	
2.	If there are changes to your contract since the previous attestation (or if the contract has not been previously attested), then the 'Request Contract Review' button will be visible. Click 'Request Contract Review' to submit the contract to be reviewed. Note: Your contract changes should be pushed to master before raising this request.	
3.	A confirmation window will appear, where you can confirm the details of the file being signed. You can enter additional details to provide more information on the changes being made. An email notification will be sent to the chapter leads and architects of the domain. If your domain does not have a chapter lead, the request will be sent to the Mesh Central Governance team.	

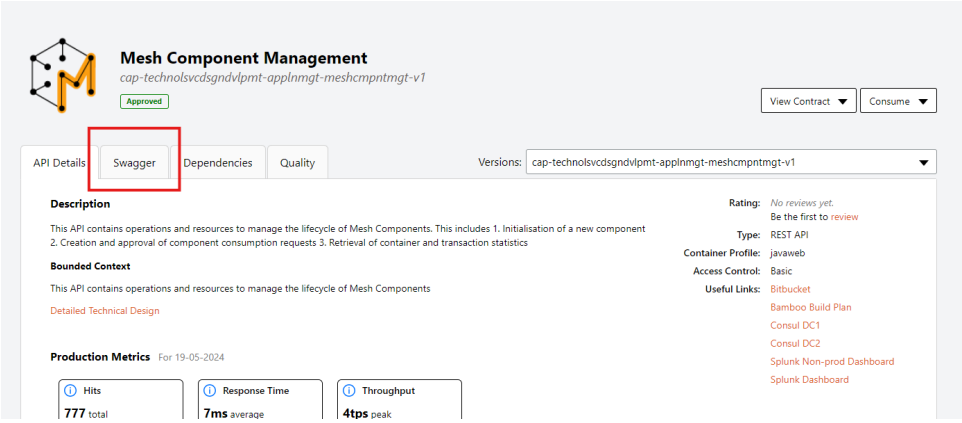
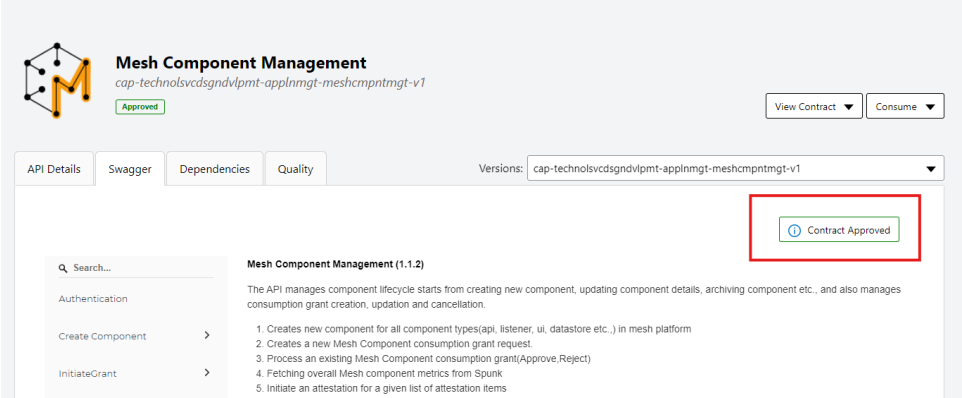
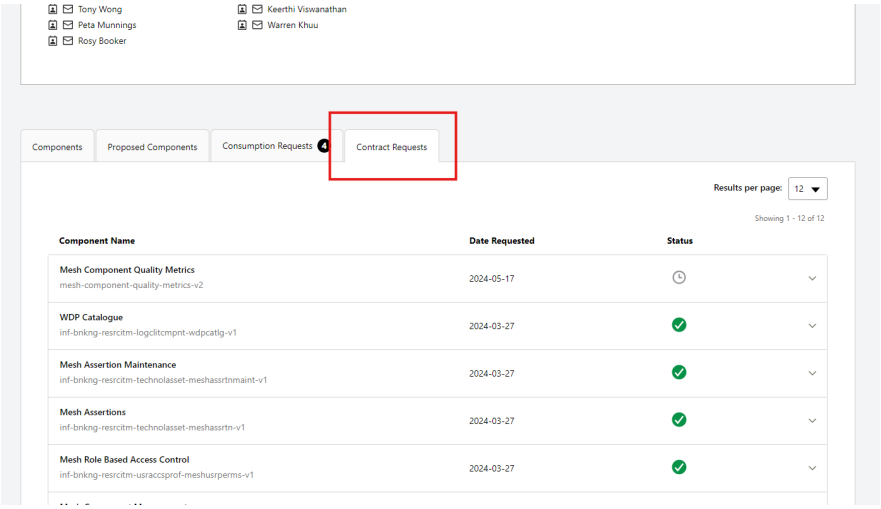
Approving API Contract Attestation

#	Action	Comment
1.	Navigate to your domains page in Mesh Connect. Select the 'Contract Requests' tab.	

		
2.	Pending contract attestation requests will be displayed at the top of the list, with a grey clock as the status. Click on the request that you want to view. Alternatively, the notification email contains a link to the request	
3.	If you are a chapter lead, you will be able to see buttons to 'Approve' or 'Reject' the request. The 'View Contract' dropdown provides various options for viewing the contract changes. It is recommended to review the changes before providing your attestation. Clicking 'Approve' will bring up a confirmation popup. Clicking 'Reject' will also require you to enter a comment, to provide a reason for the rejection.	

Viewing API Contract Attestations

#	Action	Comment
1.	To determine if the current contract is signed: Navigate to your component details page in Mesh Connect. Select the Swagger /OpenAPI tab.	

		
2.	If the contract file that is on your master branch is already signed, the 'Contract Approved' button will be displayed. You can view the commit details and history by clicking the button. If the button displays 'Contract Rejected', this means the chapter lead has rejected your contract, and you will need to make changes before you can submit another request. You can click the button to view any comments from the chapter lead.	
3.	Alternatively, you can view the entire history of contract attestations: Navigate to your domains page in Mesh Connect. Select the 'Contract Requests' tab. The list of previously attested contracts will be displayed when a green tick for the status.	

References

Sample API Contracts

Questions & Answers

Q: Which feature branch is the openapi.yaml / swagger.yaml being attested in?

A: The master branch of your repository will be used for attestations.

Q: Is the swagger-review branch still required?

A: While we still recommend you use a separate feature branch for contract reviews within your domain (such as swagger-review), the attestation process only uses the openapi.yaml / swagger.yaml on your master branch.

Q: My .minisig is not being updated / not being generated in my repository. What is happening?

A: The .minisig is not used as part of the contract attestation process. The presence of an approved contract attestation request (see Viewing API Contract Attestation) will be validated instead.

Q: When deploying to SIT/UAT, I get an error that my contract has not been signed.

A: The validation during deployment will check for either an approved contract attestation request, or a .minisig. This is to support existing contracts that have been signed via the signing pipeline, and have not yet been attested in Mesh Connect.

Q: Which component types are supported?

A: Currently, APIs, Service Proxies, Partner Proxies and MFEs are supported.

Q: Do I need to rebuild my component after contract attestation?

A: Previously, if there were changes to your contract, you would need to sign the contract, and then rebuild your component. If you attest your component through Mesh Connect, you only need to redeploy your component, without rebuilding.

API Contract Mandatory Checks

Note that the following checklist applies to both APIs built on the Westpac Mesh platform as well as proxies for APIs delivered as part of third party packages. Where specific items apply only to the former or the latter then this is indicated for each specific item.

- Info object - modify all placeholder text from the template
  - version -- ensure that the version number is updated appropriately in line with the Semantic Versioning principles
  - description -- **Must** clearly explain the purpose and scope of the API
  - x-goldenVersion -- Must ensure that the latest version of the golden template used for writing the Spec
- Host must be set to 'localhost:8080'
- Base Path must conform to naming standards and be confirmed via API Naming Governance (usually before the Swagger spec is reviewed)
- Schemes must be set to 'http'
- Security must be of type: basic
- The following items only apply to Westpac Mesh APIs (internal and partner)
  - /healthcheck resource must be defined exactly as provided in the template
- The following items only apply to Westpac Mesh internal APIs
  - All global header parameters prefixed with 'x-' which are marked as 'required: true' must be included in all operations
  - brandSilo, when present, must be passed as either a query or body parameter, depending on the http verb being used. Occasio nally, it might be passed as a path parameter.
  - Only operations (verb+resource) approved by API Naming Governance **must** be used
  - Operation payload elements should be directly related to the resource
  - All responses must be wrapped inside '**data**' object.
  - All operations **must** use appropriate HTTP response codes as defined in the Detailed Design Guidelines and exemplified in the Golden Swagger Template
  - All error codes defined in the API must be documented as per the Golden Swagger Template
  - Abbreviations like ABN, ACN etc need to be exposed in capital letters
  - Appropriate Common Data Types are used where applicable
  - Date, time and timestamp fields must be declared as a string type and qualified with the formats defined by the Swagger (OpenAPI) Specification. i.e. RFC3339 (an Internet profile of ISO8601)
    - date format - date (RFC3339 full-date)

- timestamp format - date-time (RFC3339 date-time)
- time (only) format - time (RFC3339 full-time) - this format is not prescribed by the Swagger Specification but follows the intent of this specification

Metadata Yaml File Checklist - Metadata Yaml files

Component.yaml

- brandSiloSupported -- Must identify which brand silos are explicitly supported by the API
- layer --Must correctly identify the layer applicable to this API
- applicationId - Must identify and record the app Id from alfabet
- previousVersion - If previous version is available then the previous version component name must be filled.
- componentLifecycleState - Be default this is approved. If the component deployed in production it should be updated to 'active' . If 'deprcated' then this should be updated.
- tags - Add tags for quicker search in MESHConnect

Support.yaml

- supportTeam -- Must have Remedy support group name and application Id
- vendor -- Must be provided where API is not hosted on (ie. a proxy for a third party API)
- partner - Fill this section if this is a partner API

Service-level.yaml

- Under SLA ,record the maxTPS for your API
- technicalDebt -- should be documented where applicable
- detailedTechnicalDesignURL -- Must update with technical documentation (confluence etc)

(DEPRECATED) Governance and Swagger Endorsement

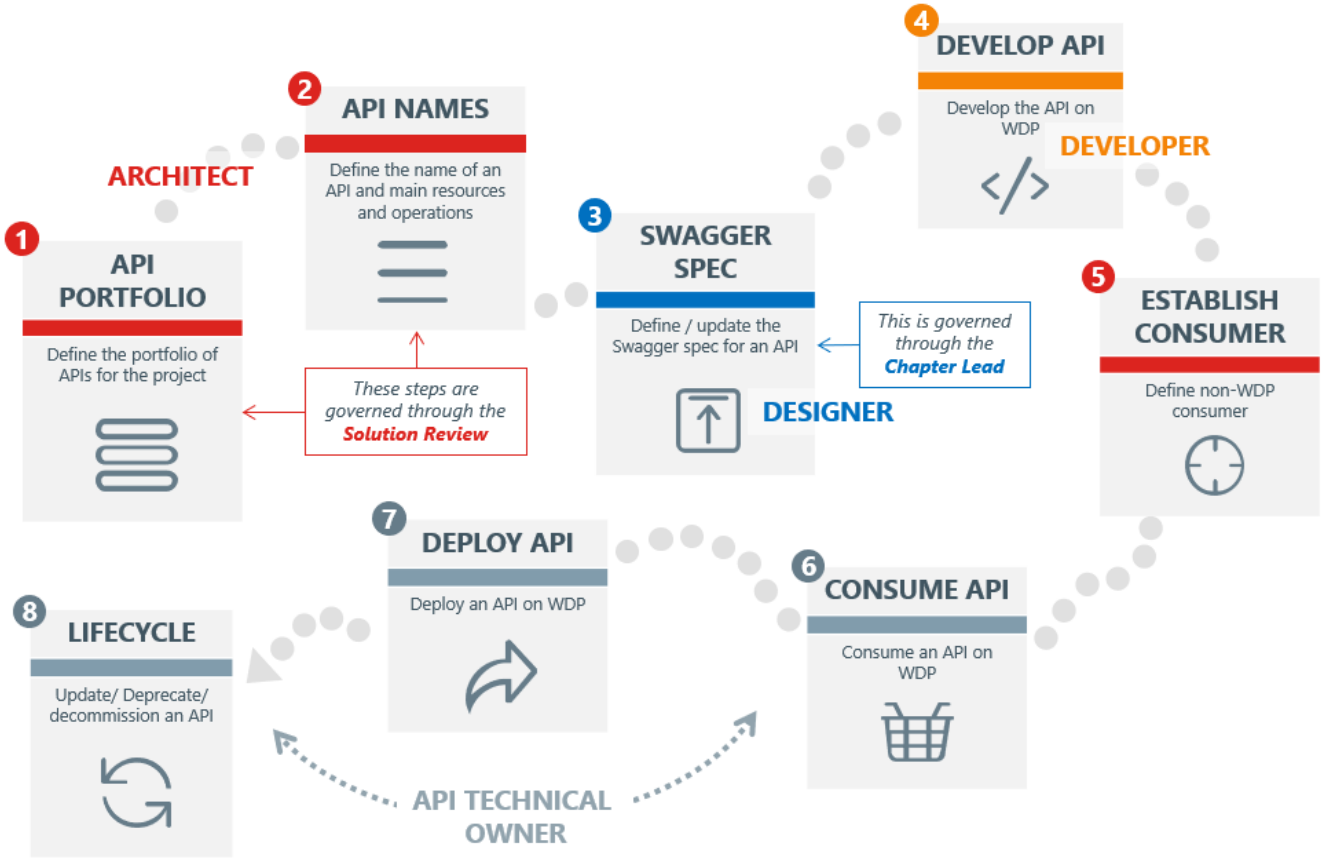
Mesh Governance

The goal of Mesh Governance is to govern Mesh components such as API , Topic , Listeners , UI etc by providing a clearly defined and proactive point of accountability for practical, pragmatic and effective oversight of the Mesh Platform strategy and capability for the Westpac Group.

Purpose

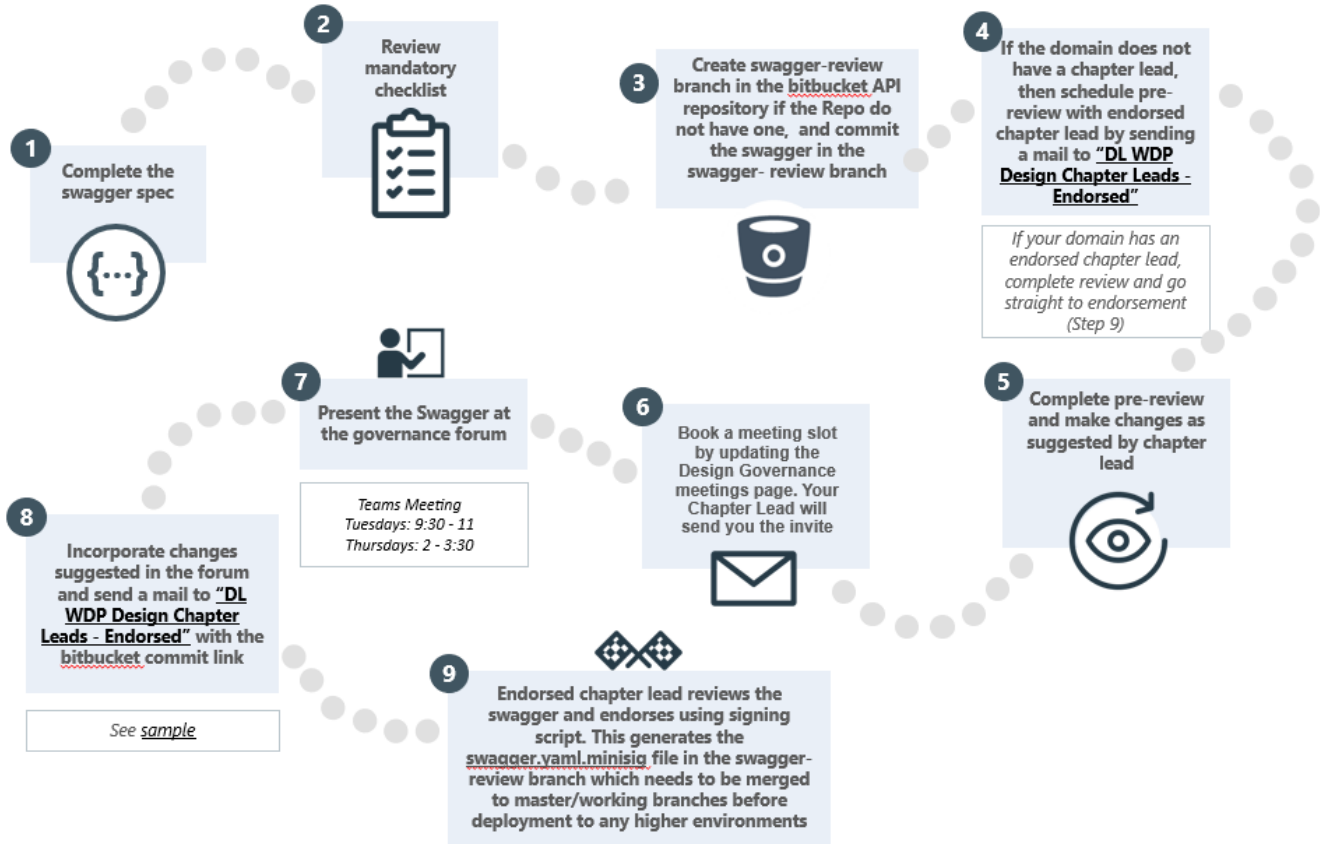
- To drive ‘best practice’ Governance by ensuring alignment with the IT Governance, Group IT strategy, architecture & design direction
- Ensure that project quality assurances reflect the required outcomes for key projects
- Ensure the suitability of Governance platforms and infrastructure, including technology and tool selection, and proactively assure performance of delivered capability
- Prepare, review and gain IT governance approval of Governance principles, policies, standards and patterns
- Drive evolution of Governance towards a maturity level of sustained business value and agility
- Identify and raise IT Risk related risks to appropriate IT Risk forum for assessment and resolution

API Governance Points



1. The Architect defines the portfolio of APIs for the project. The registry of portfolios can be found here.
2. As part of Solution Review (Naming Governance), the APIs and resources names are defined. The list of approved APIs can be found here.
3. The Designer now defines the Swagger specification. This is then governed by their Chapter Lead or through Swagger Governance.
4. The Developer now generates code from the Swagger and begins development. Note that this step can begin before the Swagger is approved, but some rework may be required if changes the Swagger are recommended.
5. Any Consuming Applications should be identified and defined by the Architect.
6. Consumption requests on any Westpac Mesh APIs are raised as part of a separate process outlined here.
7. Deployment of the API to lower environments can be performed by the Developer through Westpac Mesh Developer Tools. Deployments to Production are managed by the Westpac Mesh BAU team.
8. The lifecycle of the API is managed by the Domain Owner.

API Contract Endorsement - Governance Process

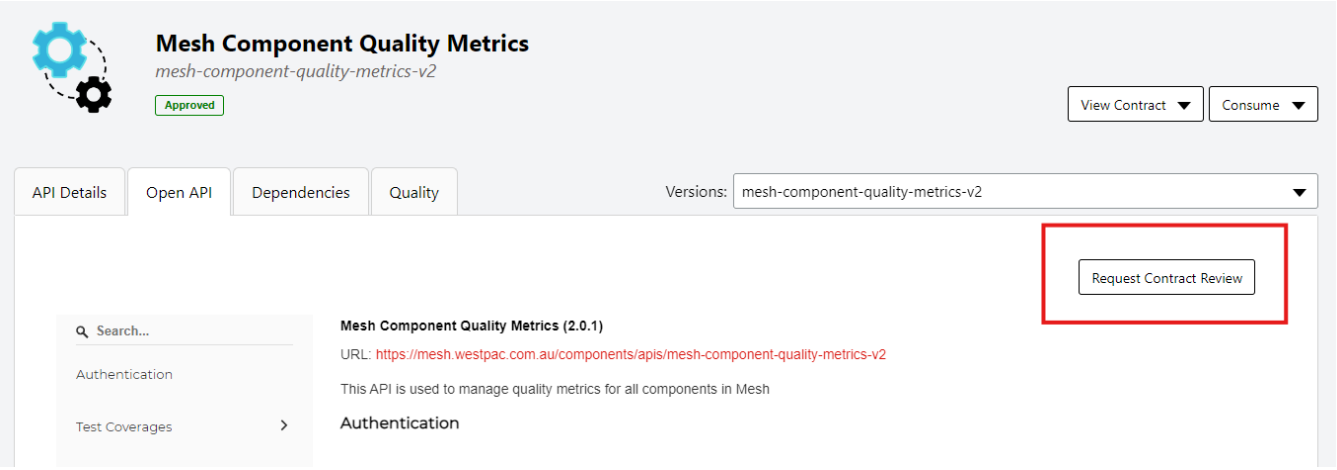


A digital signature is required to promote an API to UAT, SVP and Production environments. This signature indicates that your API is well defined and conforms to the Mesh design standards.

This process is for domains that do not have an endorsed chapter lead. You will be required to present your changes at the Mesh Design Governance meeting, where a forum of Chapter Leads will provide feedback on your contract design.

Book here: Mesh Design Governance Meetings

API Contract Attestation - Mesh Connect



This alternative process for API contract attestation is available to domains with an endorsed chapter lead. The attestation is replacement for the digital signature, and will allow you to promote an API to UAT, SVP and Production environments.

This is automated through Mesh Connect, making it easier to manage contract reviews.

Full instructions: How to Request and Approve API Contracts

Governance Meetings

Futher reading: Mesh Governance

5 Design Guidelines

API Design Guidelines

Topics

- RESTful API Overview
- Resource Naming & URIs (Resource Naming URIs, Resource Naming Anti-patterns, Pluralisation)
- Use of HTTP Verbs (GET, PUT, POST, DELETE, OPTIONS, TRACE, PATCH, HEAD)
- Paging, Sorting & Filtering
- Response Status Codes
- Data Representation in Requests & Responses
- Versioning of APIs

RESTful API Overview

Terminology

- API - a collection of resources with associated actions (HTTP verbs) providing a cohesive suite of functionality
- Resource - an object/entity which is the central focus of the intent of an action to be applied
- Method/Action - the HTTP verb to be applied to the resource
- Endpoint/Operation - the combination of method/action and the resource - roughly equivalent to an operation on a SOAP web service

RESTful APIs

There is a good overview of the REST architectural style at: http://www.restapitutorial.com/lessons/whatisrest.html

Here are some key points for successfully offering REST APIs:

- Use HTTP Verbs appropriately (align with semantics in RFC7231)
- Provide sensible resource names
- Use HTTP response codes to indicate status
- Create resources at the right level of granularity

Resource Naming & URIs

Summary

- Use nouns and not verbs in resource names
- Use plural nouns in all cases for collections, i.e.

    - /customers instead of /customer
- Differentiate between the collection resource and the singular resource, i.e. everything is a resource. Rethink interactions in terms of resource states, not service operations
    - /customers is the collection, /customers/1234 is a single entity
- Define resources at the right level of granularity
    - don't define them so coarsely that they encompass all information within a context (eg. the world)
    - don't define them so finely that a consumer would need to interact with multiple resources in order to collate the required information
- Minimise hierarchies past 3 levels: /<resource>/<identifier>/<resource>, i.e.
    - /customers
    - /customers/1234
    - /customers/1234/transactions
    - /transactions/1111

Resource Naming Structure

An API is a collection of verb actions on a resource set, with supporting data representations. Guidance for the design method, which will assist in identifying resources from a consumer use case and behaviour model, is given in the Delivery Methodology.

Resource names should be nouns. It is acceptable to use a verb-like adjective with the nouns as long as it is still clear that an object, a thing, is being identified.

Each resource should have its own URI. There should be no sharing of address. The resource structure should be predictable (it makes sense) and hierarchical within the confines of usability (it provides the ability to manipulate sub-structures without confusing the user or making parsing complex). Relationships should make sense, if the design method is followed properly.

Resources are generally plural collections, with an identifier as part of the URI to identify a unique resource.

Resources should not bleed information about internal data structures and types, or processes.

Resources can be decomposed into sub-resources (subsets of the primary resource) but attributes of the resource can only exist in one sub-resource. Sub-resources are most often used when updating a subset of the resource's attributes. Refer to the section below regarding the use of the PUT verb for more information.

A more detailed description of resource and endpoint structuring patterns is described in Pattern 19: API Endpoint Design Patterns.

Some excellent external articles on resource naming are shown below:
http://www.restapitutorial.com/lessons/restfulresourcenaming.html
https://www.thoughtworks.com/insights/blog/rest-api-design-resource-modeling

Resource URI Examples

To insert (create) a new customer in the system use, for example:
POST /customers

To read a customer with Customer ID# 33245, for example:
GET /customers/33245

The same URI would be used for PUT and DELETE, to update and delete, respectively.

Where a resource represents a process, such as the fulfilment of a new account for a customer, you could initiate that process as follows:
POST /accountFulfilment

Fully qualified URI Structure

All URIs for RESTful APIs should adhere to the following format:

{base url} / {layer} / {Domain (1..1)} / {Subdomain (0..2)} / {API Name (1..1)} / {version (1..1)} / {resources (1..1)} / { sub-resources (0..1)}
It is recommended that a maximum of 8 resources be contained within each API.

Example fully qualified URI: http://api.westpac.com.au/inf/parties/customers/v1/customerSearch

Use of HTTP Verbs

Summary

The HTTP verbs comprise a major portion of our "uniform interface" constraint and provide us the action counterpart to the noun-based resource. The primary or most commonly used HTTP verbs (or methods, as they are properly called) are POST, GET, PUT, DELETE. These correspond to create, read, update, and delete (or CRUD) operations, respectively. PATCH can be used in place of PUT where a partial update (subset of fields) of a resource is required. Note that POST can also be used as a general purpose 'do stuff' action. HEAD is the final HTTP verb currently supported for Westpac Mesh APIs and is similar to GET but it MUST NOT return a response payload. It is used to retrieve metadata or check service availability for the resource.

Note: The permitted response codes, and what they mean, for each verb are described in the Response Status Codes section below.

We summarise the allowed use of HTTP verbs as follows:

- POST for unsafe to repeat operations (such as creating data or starting jobs)
- GET for safe to repeat (and potentially cache) queries
- PUT for safe to repeat operations (such as updating data)
- DELETE for safe to repeat operations that remove resources (second repeat is usually an error)
- PATCH for unsafe to repeat update operations
- HEAD for safe to repeat service availability checking, response payload size checking, and checking staleness of cached data

The following HTTP specs describe the semantics for these verbs which we honour on Westpac Mesh.

HTTP Core Semantics

HTTP PATCH Method

JSON Merge PATCH

Table of permitted verbs and resource types

HTTP Verb	CRUD equivalent	Entire Collection (e.g. /customers)	Specific Item (e.g. /customers/{id})
POST	Create	201 (Created) Response to include the identifier for the resource created to allow a subsequent GET to be requested	405 (Method Not Allowed)
GET	Read	200 (OK), list of customers. Use pagination, sorting and filtering to navigate big lists.	200 (OK), single customer 404 (Not Found), if ID not found or invalid
PUT	Update /Replace (Create is also possible)	405 (Method Not Allowed), unless you want to update /replace every resource in the entire collection in which case the outcome would be: 204 (No Content) 200 (OK), only if useful data can be returned	204 (No Content) 200 (OK), only if useful data can be returned 404 (Not Found), if ID not found or invalid Note. Technically, PUT can be used to create a resource if the primary key (id) is chosen by the Client instead of the Server but this is an uncommon case. PUT can thus be used where an endpoint can provide an insert or update, typically referred to as an upsert, but the primary key of the resource MUST be known by the Client.
DELETE	Delete	405 (Method Not Allowed), unless you want to delete the whole collection (often not desirable) in which case the outcome would be: 204 (No Content) 200 (OK), if useful data can be returned	204 (No Content) 200 (OK), if useful data can be returned 404 (Not Found), if ID not found or invalid
PATCH	Update	204 (No Content) 200 (OK), only if useful data can be returned Note. PATCH cannot be used to create or delete an item within the collection as the body of the PATCH request contains only the representation of the resource but no explicit instruction as to the action to be applied - it will always be treated as change.	204 (No Content) 200 (OK), only if useful data can be returned 404 (Not Found), if ID not found or invalid
HEAD	Health-check Read	200 (OK) but without response body	200 (OK) 404 (Not Found)

Table of permitted verbs and request parameter types

HTTP Verb	Header+	Path	Query	Body
POST	Permitted	Permitted	Permitted*	Permitted
GET	Permitted	Permitted	Permitted	Not permitted
PUT	Permitted	Permitted	Permitted*	Permitted
DELETE	Permitted	Permitted	Permitted	Not permitted
PATCH	Permitted	Permitted	Permitted*	Permitted
HEAD	Permitted	Permitted	Permitted	Not permitted

Key

+ headers should only be used to transport non-functional information (eg. auditing). Westpac Mesh has defined a set of standard headers to be used and APIs should not introduce their own headers without discussion with Westpac Mesh architecture.

* should not contain resource content (this should be in the body) but may be used to provide context or hints for processing

POST

POST is an unsafe operation in that it typically creates a new resource (or may initiate a new job/process). Each resource will be new, so the result is effectively different each time as a new identifier would be returned in the data payload (as a rule). When creating new resources the most common use case is the creation of a single new resource, however it is acceptable to request creation of multiple resources when required (ie. sending an array of items to be created). Note that when requesting creation of multiple new resources the entire request should be treated as being atomic in that all resources will be created or none will be created otherwise it will be difficult to form a response indicating which of the requested objects failed to be created.

Example:

POST /customers

POST request

```
POST /customers HTTP/1.1
Content-Type: application/json

{
    "firstName": "Samuel",
    "lastName": "Johnson",
    "middleName": null,
    "dateOfBirth": "1998-07-03",
    "gender": {
        "code": "M"
    },
    "status": {
        "code": "A"
   }
}
```

POST response

```
HTTP/1.1 201 Created
Content-Type: application/json

{
    "data": {
        "customerId": {
            "id" : "12345",
            "idScheme": "CustomerInternalId"
        }
    }
}
```

GET

GET is to be used for safe to repeat queries. The use of GET must not lead to a change of resource state, but it is valid for resource state to change between queries due to other impacting factors.

The use of GET shall not require any data payload for normal query operations. Use of query parameters is the correct way to search for data. If there is a concern that PII may be in the URI and it is desirable to avoid this due to server log caching concerns then either:

- PREFERRED: Abstract the PII into the user's identity and establish the data as part of the authentication process.
- BACKUP OPTION: Use a POST to create a query request resource and return a response ID, then use GET against that ID to obtain the results. *Note this is also a pattern for queries which may take time to complete.*
- NOT PREFERRED: Use a POST with query data in the body and return a response set to this call.

Examples:

GET /customers/12345

GET request: single resource

```
GET /customers/12345 HTTP/1.1
Content-Type: application/json
```

GET response: single resource

```
HTTP/1.1 200 Ok
Content-Type: application/json

{
    "data": {
        "customerId": {
            "id" : "12345",
            "idScheme": "CustomerInternalId"
        },
        "firstName": "Samuel",
        "lastName": "Johnson",
        "middleName": "Andrew",
        "dateOfBirth": "1998-07-03",
        "gender": {
            "code": "M"
        },
        "status": {
            "code": "A"
        }
    }
}
```

GET /customers

GET request: collection resource

```
GET /customers HTTP/1.1
Content-Type: application/json
```

GET response: collection resource

```
HTTP/1.1 200 Ok
Content-Type: application/json

{
    "data": {
        "customers": [
        {
            "customerId": {
                "id" : "12345",
                "idScheme": "CustomerInternalId"
            },
            "firstName": "Samuel",
            "lastName": "Johnson",
            "middleName": "Andrew",
            "dateOfBirth": "1998-07-03",
            "gender": {
                "code": "M"
            },
            "status": {
                "code": "A"
            }
        },
        ...
        {
            "customerId": {
                "id" : "67890",
                "idScheme": "CustomerInternalId"
```

```
            },
            "firstName": "Samantha",
            "lastName": "Jones",
            "middleName": Louisa,
            "dateOfBirth": "1985-10-23",
            "gender": {
                "code": "F"
            },
            "status": {
                "code": "A"
            }
        }
        ]
    }
}
```

PUT

PUT is to be used for creating or updating the state of the resource being called. It should be possible to make the same call against a resource using a PUT and the outcome on resource state to be the same (idempotent), assuming no other agent impacts the resource between calls.

The use of PUT for update operations creates some complexity. PUT is meant to replace the *entire* resource with a new representation, however many organisations use PUT for partial updates by only passing in the resource attributes to change. This approach however would not guarantee idempotency (repeatability) and we do not support this approach on Westpac Mesh - PUT must be used to replace the *entire* resource.

The approaches available to support *partial* updates, listed in order of most preferred to least preferred are:

- Use PATCH to update the subset of fields required. Note that you cannot use PATCH to insert or delete items within a nested collection contained inside the resource; you will need to use POST AND DELETE respectively on a sub-resource.
- Use POST on the primary resource and append '/do/*verb*' to the URI. eg. POST /customers/33245/do/terminate. This approach is appropriate to encapsulate a discrete action to be applied against the resource, equivalent to creating a new HTTP verb to describe the action. However, it is not appropriate to use *'/do/update'* to support partial resource updates as PATCH should be used for this purpose.
- Use PUT on a sub-resource of the primary resource which represents the collection of fields to be updated - all fields of the sub-resource MUST be supplied. However, we don't want to end up with a design where every field becomes its own sub-resource. Where the sub-resource contains a collection of fields it needs to be named in an intuitive fashion. Note: each attribute or property of the resource can only be present in ONE sub-resource.
- Use POST to a new resource, which is used only for updates, that is named similarly to the primary resource. eg. POST /customerUpdates. This option is suitable where the unique identifier for the resource to be updated cannot be described via the URI. This could occur where the provider system (or API) does not expose the unique identifier or the resource contains a nested collection and there is a desire to insert, update or delete item(s) in the collection along with scalar attributes of the resource within a single endpoint.
- Use DELETE to remove the resource, then use POST to (re)create the resource.

Note. It is permissible to apply PUT to a collection resource but note that it is expected that you will replace ALL items within that collection. If you want to selectively update items within the collection use PATCH instead.

Examples:

PUT /customers/12345

PUT request: single resource

```
PUT /customers/12345 HTTP/1.1
Content-Type: application/json

{
    "firstName": "Samuel",
    "lastName": "Johnson",
    "middleName": null,
    "dateOfBirth": "1998-07-03",
    "gender": {
        "code": "M"
    },
    "status": {
        "code": "A"
   }
}
```

PUT response: single resource

```
HTTP/1.1 204 No Content
Content-Type: application/json
```

PUT /customers (NOT RECOMMENDED !)

PUT request: collection resource

```
PUT /customers/12345 HTTP/1.1
Content-Type: application/json

[
    {
        "customerId": {
            "id" : "12345",
            "idScheme": "CustomerInternalId"
        },
        "firstName": "Samuel",
        "lastName": "Johnson",
        "middleName": null,
        "dateOfBirth": "1998-07-03",
        "gender": {
            "code": "M"
        },
        "status": {
            "code": "A"
        }
    },
    {
        "customerId": {
            "id" : "67890",
            "idScheme": "CustomerInternalId"
        },
        "firstName": "Samantha",
        "lastName": "Grey",
        "middleName": Louise,
        "dateOfBirth": "1985-10-23",
        "gender": {
            "code": "F"
        },
        "status": {
            "code": "A"
        }
    }
]
```

PUT response: collection resource

```
HTTP/1.1 204 No Content
Content-Type: application/json
```

DELETE

DELETE is used to completely remove a resource instance. This is a safe operation in that a call can be made multiple times and only the targeted resource should be deleted, however note that a second call should be an error (resource not found) if the DELETE has been completed.

Note. A request body cannot be sent when using the DELETE verb.

Examples:

DELETE /customers/12345

DELETE request: single resource

```
DELETE /customers/12345 HTTP/1.1
Content-Type: application/json
```

DELETE response: single resource

```
HTTP/1.1 204 No Content
Content-Type: application/json
```

DELETE /customers (NOT RECOMMENDED !)

DELETE request: collection resource

```
DELETE /customers HTTP/1.1
Content-Type: application/json
```

DELETE response: collection resource

```
HTTP/1.1 204 No Content
Content-Type: application/json
```

PATCH

PATCH is used to update a subset of attributes for a resource. This is an unsafe operation in that the state of a resource may be different after applying the same PATCH request multiple times due to simultaneous updates applied by other parties. There are various PATCH request formats in use within the industry. Westpac Mesh has adopted the JSON Merge PATCH format as it supports the same request format as used for POST and PUT verbs.

Note. The target resource URI may identify a single object or a collection of objects. If it identifies a collection of objects only existing objects within the collection can be updated - you cannot insert or delete objects within the collection using PATCH.

Examples:

PATCH /customers/12345

PATCH request: single resource

```
PATCH /customers/12345 HTTP/1.1
Content-Type: application/merge-patch+json

{
   "lastName": "Johnson",
   "middleName": null
}
```

PATCH response: single resource

```
HTTP/1.1 204 No Content
Content-Type: application/json
```

PATCH /customers

PATCH request: collection resource

```
PATCH /customers HTTP/1.1
Content-Type: application/merge-patch+json
```

```
[
   {
      "customerId": {
         "id": "1234",
         "idScheme": "CustomerInternalId"
      },
      "lastName": "Johnson",
      "middleName": null
   },
   {
      "customerId": {
         "id": "5678",
         "idScheme": "CustomerInternalId"
      },
      "lastName": "Warren",
      "firstName": "Jennifer"
   }
]
```

PATCH response: collection resource

```
HTTP/1.1 204 No Content
Content-Type: application/json
```

HEAD

HEAD is to be used for safe to repeat queries. The use of HEAD must not lead to a change of resource state, but it is valid for resource state to change between queries due to other impacting factors. Note that as HEAD is used for probing API availability, response payload size, and cache expiry, the actual resource will NOT be returned.

Examples:

HEAD /customers/12345

HEAD request

```
GET /customers/12345 HTTP/1.1
Content-Type: application/json
```

HEAD response

```
HTTP/1.1 200 Ok
Content-Type: application/json
```

Paging, Sorting and Filtering

Please refer to Pattern 9: Paging, Sorting and Filtering

Response Status Codes

Summary

Most HTTP common response codes are generally well understood by API consumers, such as 404 – Not Found. However, others are less commonly understood, and it is important designers and implementers are consistent in how they are used in order to breed a consistent familiarity for API consumers. A good accessible resource for HTTP codes and their meanings is in Wikipedia: https://en.wikipedia.org/wiki/List_of_HTTP_status_codes

A summary table is provided below showing which codes are in use on Westpac Mesh. If the code is not in this table, it is not to be used as a response code. If a use case is not covered and requires a new response code, then propose through program architecture. Note that only a few combinations are invalid due to the filtering down of response status codes.

Code		HTTP Verb					

	Status	POST	GET	PUT	DELETE	PATCH	HEAD
200	Ok	Y	Y	Y	Y	Y	Y
201	Created	Y	N	Y	N	N	N
202	Accepted	Y	N	Y	Y	Y	N
204	No Content	Y	N	Y	Y	Y	Y
303*	See Other	Y	Y	N	N	N	N
400	Bad Request	Y	Y	Y	Y	Y	Y
401	Unauthorised	Y	Y	Y	Y	Y	Y
403	Forbidden	Y	Y	Y	Y	Y	N
404	Not Found	Y	Y	Y	Y	Y	Y
409	Conflict	Y	N	Y	Y	Y	N
500	Internal Server Error	Y	Y	Y	Y	Y	Y

* Only applicable for asynchronous processing, refer Pattern 5: Asynchronous Processing.

The following response codes have been considered for inclusion in the above list but dismissed as being unsuitable.

Code	Status	Purpose for inclusion	Reasons for exclusion
207	Multi-Status	To support partial success scenarios	• Part of the WebDAV extensions for HTTP to support distributed authoring - limit HTTP response codes to core set • Response structure is assumed to be XML • Response structure is composed of a single results object containing an array of objects indicating the success or otherwise for each item - our preference is to use the data object to return successful processing results and only use the results object to report the errors
422	Unprocessable Entity	To distinguish between syntactical and semantic client errors	• Part of the WebDAV extensions for HTTP to support distributed authoring - limit HTTP response codes to core set • 400 Bad Request can mean syntactical or semantic errors (original definition in RFC2616 only covered syntactical errors) • Little benefit in distinguishing between syntactical and semantic errors - keeps it simple for designers/developers • 400 is the generally accepted response code for client errors
501	Not Implemented	Enable the response code for Information and Capability APIs, to indicate in a resource /end-point that certain functionality is not yet implemented	• Status code indicates that the server does not support the method/end-point to fulfill the request. • Status code does not align to the use-case need, as the method/end-point exists and certain functional detail is not yet implemented. • Swagger is the primary mechanism to describe whether a feature has been implemented or not. e.g. if the endpoint is not in Swagger it is not ready for production use ; if the brandSilo is not in the list of allowed enums, then it is not ready for production use. • Swagger is also the primary mechanism for Service Discovery through Connect, hence the contract needs to be reflective of what is implemented and available features. • Considering above, Swagger should be limited to supported features. • Current use-case need is for internal APIs (information and capability) and the decision is scoped at that. If other API layers need to support the response code arises, then it will need re-assessment based on use-case.

Response Code Detailed Guidance

Further details on the codes to be used is provided below, as elaborated from: http://www.restapitutorial.com/httpstatuscodes.html and IETF.org

200 Ok

The request has succeeded. The information returned with the response is dependent on the method used in the request, for example:

- POST: a response body may be returned if required by the use case, though this should be limited to new identifiers (when creating a new resource) rather than an entire representation of the resource.
- GET: an entity corresponding to the requested resource is sent in the response.
- PUT: the typical response code returned will be 204 with no response body as there is no meaningful data to be returned. A response body may only be returned if there is meaningful data to return and in this case the response code will be 200.
- DELETE: the typical response code returned will be 204 with no response body as there is no meaningful data to be returned.
- PATCH: the typical response code returned will be 204 with no response body as there is no meaningful data to be returned. A response body may only be returned if there is meaningful data to return and in this case the response code will be 200.

- HEAD: the entity-header fields corresponding to the requested resource are sent in the response without any message-body.

This is a general status code, and the most common code used to indicate success where more fine grained success handling is not required.

201 Created

The request has been fulfilled and resulted in a new resource being created. This should only be used for PUT or POST. The resource must have been created. If creation is pending, then return 202 instead.

We will still provide a resource ID in the response body if appropriate, to support subsequent calls, but if this is not appropriate the API design must support another query mechanism for finding resources.

202 Accepted

The request has been accepted for processing, but processing has not completed. It may be subsequently rejected. We will only support this status code for DELETE, POST, or PUT. Call-back queries will be implemented using a POST, followed by a GET.

We will still provide a resource ID in the response body if appropriate, to support subsequent calls, but if this is not appropriate the API design must support another query mechanism for finding resources.

204 No Content

The request has been successfully processed, but there is no data to return. This is an appropriate response for PUT, POST, DELETE, PATCH methods where there is no meaningful data which can be returned to the caller. It is important to note that it is prohibited to return any content in the response body when providing a 204 response code. If there is meaningful data available to be returned then use a 200 response code with a body containing this data.

Use of a 204 response code is not recommended for the GET method. The caller will expect to receive a 200 response code for successful invocation of this method. If you are accessing a collection resource (eg. /customers) and apply a filter to the collection (eg. postCode=9999) which results in no matching customers found then the response should be a 200 response code and a body containing an empty array of customer objects.

Note that the 204 should not be used as a general code for HEAD type queries, as HEAD should provide the same status code response that the GET would have given, had it been made in its place.

303 See Other

The resource you have requested is not available at this URI but is available at the address identified in the Location HTTP header.

400 Bad Request

The request could not be understood due to malformed syntax or data. The consumer should not resubmit without making changes.

401 Unauthorised

Use this error code response for missing or invalid authentication token, but if bleeding through security failures is disallowed, use 404 instead. Westpac Mesh APIs should not generally expose a 401 response code as the APIs should authenticate with the backend provider at an application level rather than user level. However, the caller may receive a 401 response code if the API gateway cannot authenticate the caller using their API key and secret.

403 Forbidden

Use this error code response when the caller is not authorised to access the provider or one of its functions, but if bleeding through security failures is disallowed, use 404 instead. Westpac Mesh APIs do not generally expose a 403 response code as the APIs should be authorised with the backend provider at an application level rather than user level; this is generally referred to as coarse grained authorisation. It is assumed that the calling application will determine whether the user should be permitted to invoke a particular function of the provider. However, if your provider does authorise individual users for specific functions then it is permitted to expose a 403 response code. It is also possible that the API gateway will return a 403 if the calling application is not authorised to call this API.

404 Not Found

Used when the requested resource cannot be found or if you do not wish to disclose that a resource exists. The latter case generally occurs for public APIs where a user fails to authenticate or be authorised and you do not wish to expose that the resource exists.

If the requested resource identifies a specific resource such as a customer identified by the URI /customers/{customerId} then it is appropriate to return 404 if no customer exists with the supplied customerID.

However, if the requested resource identifies a set of objects such as a list of accounts for a specified customer as in the URI /customers/{customerId}/accounts then it is not appropriate to return a 404 where the customer does not have any accounts. Instead a 200 containing an empty array of accounts is the appropriate response. It is appropriate to return a 404 if no customer exists for the specified customerId.

409 Conflict

Use when a resource conflict would be caused by fulfilling the request. This should only be the case for PUT, POST and DELETE methods. Potential scenarios for returning a 409 are: resource duplication, optimistic lock violation, resource is not in an apppropriate state for the action.

500 Internal Server Error

This general catch-all error covers when the server-side throws an exception. Whilst there are many 500 series status codes, for an API these should almost never be thrown. Use 500 only for errors that the consumer cannot address from their end.

Data Representation in Requests and Responses

Summary of Approach

All payloads must ONLY support the MIME content type application/json. The following exceptions apply:

- application/merge-patch+json - for use with PATCH verb requests only
- other MIME types such as image/jpeg as the response where the content to be returned consists solely of that content type. However, application/xml is specifically excluded.

Fundamental principles

- All responses must return the resource data contained within a data attribute. This allows additional metadata such as paging information to also be included at the root level
- The HTTP Accept header should expect to receive application/json but can safely ignore other values
- Use camel case for all field names in JSON
- Use ISO8601 for date time formatting, i.e. 2012-01-01T12:00:00Z. Note that the date time value must be returned as UTC time not local time.
- Nest foreign key relations as this allows simple future inlining of extra information, e.g.

{ customer: { id: 1234} } vs { customerId: 1234 }

- Use standard names for common fields, i.e. id, modifiedAt, createdAt
- Errors should be returned in an error attribute for a JSON response. This should not be returned along with a data attribute.This is an array of error objects, each containing:

code – Application specific error code

message – Human readable description, but not necessarily intended for display to the end user

details – Detailed information about the error

- Use Common Data Types where applicable to ensure consistency across all Westpac Mesh APIs
- Don't echo information in the response payload which was provided in the request (body/header/path)
- Design the interface for ease of use by the consumer(s)
    - The API should provide an abstraction of the underlying provider system data models - don't just echo the provider interface in the API interface
    - Don't design the interface to satisfy a single specific consumer - design for all possible consumers
- If there are more than three levels in the JSON data structures, consider your resource granularity again.
- If a response (ignoring paging a collection) is more than a page of text when printed out, then consider again your resource granularity.
- Don't be afraid of supporting multiple query parameters. Most consumers will not use more than a few parameters at a time. The size of a generated URI is usually irrelevant to the consumer anyway. Let the consumer requirements drive this.
- Don't send fields with null data - just don't send them
- Make field and property names meaningful
- Use double quotes for all field names and values - this is generally enforced by the JSON serialiser for the language framework being used

Versioning

Westpac Mesh APIs honour semantic versioning principles in that a new major version is not guaranteed to be backwardly compatible with a prior major version, a new minor version is backwardly compatible with a prior minor version, a new patch version is backwardly compatible with a prior patch version.

Westpac Mesh APIs supports both **URL versioning** and **Media type versioning.**

URL versioning is on by default in which include the major version number in the URI which allows side by side deployment of different major versions of an API

Media type versioning is available for Open API 3.0 and is a flexible solution for versioning endpoints without creating a new major version of an API.

- It allows versioning individual API endpoints when you need it.
- You can version input and output schemas independently.

Media type versioning allows the old endpoint to be marked as deprecated and a new endpoint to be created in the same version of the API. Only two endpoint versions are allowed before a major version needs to be created. When a new major version is created , any deprecated endpoints MUST be removed.

You should consider the pros and cons of each versioning strategy, and choose the one that suits your needs and preferences. You should also follow some best practices, such as minimizing breaking changes, documenting your API versions, and deprecating old versions gracefully.

The use of JSON instead of XML as a data format for APIs provides the opportunity to make many changes in a backwards compatible fashion. The table below illustrates some typical types of changes and whether they are backwardly compatible (non-breaking) or not. Note that the term 'operation' refers to the combination of HTTP verb and URI.

Type of change	Backward compatible
Add new mandatory request field (query,request body,header etc)to an existing operation.	No
Add new optional request field (query,request body,header) to an existing operation	Yes
Add new fields/objects to the response of an existing operation	Yes
Add new operation to an existing API	Yes
Rename or delete an existing field from an operation	No
Rename or delete an existing operation	No
Change cardinality of a field from 1 to n or n to 1	No
Add new http status code	No
Add new error code	Yes
Enhance enum(s) to add more values in request	Yes
Enhance enum(s) to add more values in response	No
Change mandatory request field(query,request body,header) to optional	Yes
Change data types in request /response	No
Change / Remove enum values in request / response	No
Change/ remove existing endpoint names	No
Change / remove existing http status codes / error codes	No
Change endpoint verb - example GET to POST	No

Note then when a new major version of an API is required, the new major version MUST include all operations and associated functionality from the prior version except where this functionality is deliberately being deprecated in the new version.

Maturity Levels

Whilst being "properly" RESTful requires compliance to a number of architectural style guides and patterns, what we are trying to achieve with APIs is more important than ivory tower arguments about levels of compliance with the RESTful style guides. That said, the following link provides a good description of a maturity model for Restful APIs which takes a coarse grained approach to defining increasing levels of maturity, and how this benefits the integrating systems: http://martinfowler.com/articles/richardsonMaturityModel.html

The following table summarises the Richardson Maturity Model:

Level	Description
0	"The Swamp of POX/POJ" HTTP is used as the transport, but without using any of the web mechanisms. Effectively, this is XML or JSON remote procedure calls (Plain old XML/JSON).
1	Resources Rather than making all requests to a singular service endpoint, we now start talking to individual resources.
2	HTTP Verbs Start using the HTTP verbs as closely as possible to how they are used in HTTP itself
3	Hypermedia Controls Introduce Hypermedia As The Engine Of Application State - The message flow defines the available behaviour on the resource - "What can one do next?"

There is a lot more to maturity than just this, as one could argue that:

Level 1 should introduce some additional standards for using resources, but the key is to start treating endpoints as managing resources, not providing services, in order to allow the next step to work.

Level 2 should cover off some of the other core HTTP behavior management capabilities, especially around caching, but the key is at this level we are driving the message flow behaviour in line with how the web's client server model works.

Level 3 is nirvana and challenging to implement fully with APIs.

For Westpac Mesh APIs, we are targeting level 2 compliance and will support level 3 (HATEOAS) for use cases where it makes sense to do so. The precise design model which will be used to support HATEOAS is yet to be finalised.

References:

http://www.restapitutorial.com/lessons/whatisrest.html

http://www.restapitutorial.com/lessons/httpmethods.html

http://www.restapitutorial.com/httpstatuscodes.html

http://www.restapitutorial.com/lessons/restfulresourcenaming.html

https://www.thoughtworks.com/insights/blog/rest-api-design-resource-modeling

http://martinfowler.com/articles/richardsonMaturityModel.html

http://www.apiacademy.co/ (API Design 303: Pagination)

https://tools.ietf.org/html/rfc7231 (HTTP Semantics)

http://tools.ietf.org/html/rfc5789 (HTTP PATCH method)

HTTP PATCH formats

https://tools.ietf.org/html/rfc7396 (JSON Merge Patch)

RFC 9110: HTTP Semantics (rfc-editor.org)

Defining Responses

Supported Success Response Codes

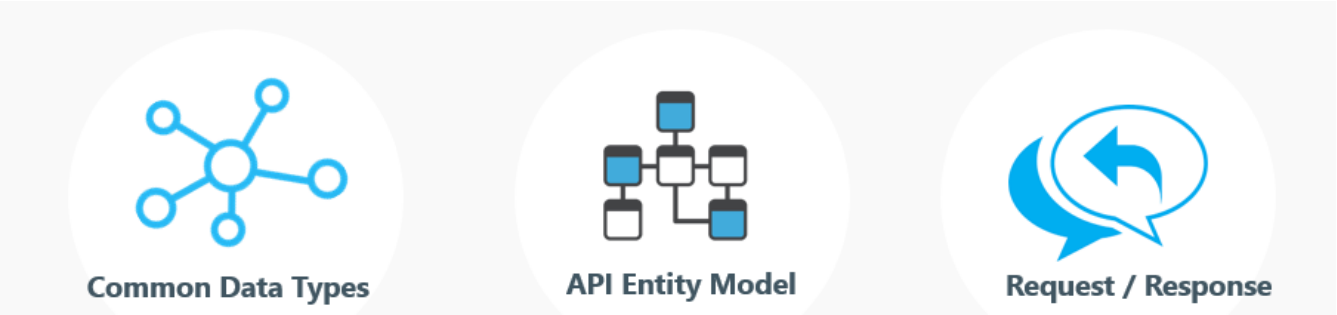
HTTP Status Code	Status	Description
200	OK	The standard response for successful HTTP requests, should contain a request body
201	Created	The request has been fulfilled, resulting in the creation of a new resource
202	Accepted	The request has been accepted for processing, but the processing has not been completed. The request might or might not be eventually acted upon, and may be disallowed when processing occurs
204	No Content	The server successfully processed the request and is not returning any content

Supported Failure Response Codes

HTTP Status Code	Status	Description
400	Bad Request	The server cannot or will not process the request due to an apparent client error
404	Not Found	The requested resource could not be found but may be available in the future, subsequent requests by the client are permissible
409	Conflict	The request could not be completed due to a conflict with the current state of the target resource. Can be used when optimistic locking is in use.
500	Internal Server Error	A generic error message, given when an unexpected condition on the server was encountered and no more specific HTTP response code is suitable

Swagger Definition Objects

Three Types of Swagger Definitions



Common Data Types

There are Westpac Mesh API resources that are common throughout most if not all APIs. This will contain the main Error definition, but can also contain other common definitions in the future.

Examples: Currency amount, balances, percentage, identifiers., time period.

- Primitive Data Types
- Common Data Types
  - Identifier
  - CurrencyAmount
  - Balance
  - TimePeriod
  - Time offsets from UTC
  - Percentage
  - Address
  - Logical Operator
  - ERD fields
  - Error Object
- Common Identifier structures
- Common Field Names
- Field Naming Conventions
  - Data Type: Boolean
  - Data Type: Date/DateTime
  - Acronyms
  - Abbreviations
- Defining Additional Information as Key / value pairs
- Importing Common Data Types

Overview

This page describes the common data types defined by Westpac Mesh and their typical usage. All APIs are expected to use these common data types where applicable. New data types will be added to this page over time as they are discovered.

Primitive Data Types

Westpac Mesh API primitive data types are based on the Swagger Specification v2 that can be found here, but with some exceptions as indicated in the table below.

Primitives have an optional modifier property `format`. Swagger uses several known formats to refine the data type being used.

Common Name	`type`	`format`	Comments
float	string	regex	There is a known issue in JSON serialisation/deserialisation with respect to float and double numbers which causes a loss of precision. Therefore, all fractional numbers will be declared as strings with a format describing the actual number format.
double	string	regex	As per above.
date	string	date	As defined by `full-date` - RFC3339 - format YYYY-MM-DD

dateTime	string	date-time	As defined by `date-time` - RFC3339 - format YYYY-MM-DDThh:mm:ss.fffZ Note: Use ISO8601 for date time formatting, i.e. 2012-01-01T12:00:00Z. Note that the date time value must be returned as UTC time not local time
time	string	time	As defined by `full-time` - ISO8601 - format hh:mm:ss.fffZ
integer	integer	int32	
long	integer	int64	
string	string		
boolean	boolean		
enum	string		A fixed list of possible values. If this field is used in conjunction with the defaultValue field, then the default value MUST be one of the values defined in the enum. All values in an enum must adhere to the specified **`type`**. ***enums* are not properly defined when *spaces* appear in the string .** So replace space with underscore or words without a seperator. eg:'Not Selected' must be defined as 'NOT_SELECTED' or not_selected or Not_Selected or NotSelected Name enum values in business terms not code. Do not define enums as 0,1 etc . It should be named in proper business terms. **Keys things:** • Dots are not allowed in Enum values. • Use Upper cases for abbreviations/acronyms • Use _ instead of spaces/dots or combine the words • Define enum values in business terms.

Common Data Types

Data Type	Description	Composition	Comment	Example	yaml
Identifier	General Identification structure to capture the identification scheme and the identification value to uniquely reference a resource	"id": The string containing the identifier value "idScheme": The identification scheme used	Refer to separate section on common identifier structures	`{ id: "12345678901", idScheme: "CustomerInternalId" }`	Below syntax can be used if identifier is used in body : `CustomerId: type: object required: - id - idScheme properties: id: type: string idScheme: description: > Scheme of the Id eg: CustomerInternalId /CustomerNumber /PartyUID /ExternalSystemCustId. - CustomerInternalId is Customer Id from the Source System eg. GCISKey or CISKey - PartyUID - Party Universal ID , generated in GCM. - CustomerNumber is Internet Bank Client id. - ExternalSystemCustId is Customer Id from External Source System. type: string enum: - CustomerInternalId`

					- CustomerNumber - PartyUID - ExternalSystemCustomerId
CurrencyAmount	A numerical amount used to represent money	“currencyCode”: String containing the three character [ISO 4217] code “amount”: **String** value of the total number of units, default is to 2 decimal places, except for currencies that require more as per ISO 4217, i.e. Chile. Negative amounts must be prefixed with minus sign. ('-')	Currency code is implied to be ‘AUD’. Might not be required internally, but will be required for Open APIs in future. As at March 2018, the maximum number of decimal places in use for any international currency is 4 Note, ‘$’ or any other currency indicator MUST not be returned in amount	{"currencyCode": " AUD", “amount": "123.45"}	currencyAmount: description: A qualified amount and denomination of a monetary value. type: object properties: amount: type: string pattern: '^[-]?\d{1,13}\.\d{1,4}$' currencyCode: description: The ISO4217 currency code of the currency type: string

Balance	Based on CurrencyAmount with addition of 'position' to indicate if the balance is in Credit (CR) or Debit (DR)	“currencyCode”: String containing the three character [ISO 4217] code
		
		“amount”: **String** value of the total number of units, default is to 2 decimal places, except for currencies that require more as per ISO 4217, i.e. Chile.
		
		"position": Indicates if the amount in CR/DR
			Currency code is implied to be ‘AUD’. Might not be required internally, but will be required for Open APIs in future.
			
			As at March 2018, the maximum number of decimal places in use for any international currency is 4
			
			Note, ‘$’ or any other currency indicator MUST not be returned in amount
				"currentBalance": {
				"currencyCode": "AUD",
				"amount": "123.45",
				"position": "CR"
				}
					currentBalance:
					description: balance amount details.
					type: object
					properties:
					amount:
					type: string
					pattern: '^\d{1,13}\.\d{1,4}$'
					currencyCode:
					type: string
					position:
					type: string
					enum:
					- CR
					- DR

Time Period	A duration of time expressed in terms of the number of units of time, such as years, months, days		Expressed according to ISO8601 duration format, as follows: P[n]Y[n]M[n]DT[n]H[n]M[n]S or P[n]W where • P, Y, M, W, D, T, H, M, and S are designators for each of the date and time elements • [n] is replaced by the number of units for each date and time element • P is the duration designator (for period) placed at the start of the duration representation • Y is the year designator that follows the value for the number of years • M is the month designator that follows the value for the number of months • W is the week designator that follows the value for the number of weeks • D is the day designator that follows the value for the number of days • T is the time designator that precedes the time components of the representation • H is the hour designator that follows the value for the number of hours • M is the minute designator that follows the value for the number of minutes	"fixedLoanTerm": "P5Y" "expectedWaitTime": "PT1M"	remainingTerm: description: Term for this liability. Term value sent as period E.g. P24M/P2Y type: string pattern: '^P[\d]Y[\d]M$' If returned only in years then use pattern as , 'P[n]Y' If returned only in months , then use 'P[n]M' "PT1M" indicates the duration of one minute

			• S is the second designator that follows the value for the number of seconds		
Time offsets from UTC	An offset from UTC represented using a string.		Negative UTC offsets describe a time zone west of UTC±00:00, where the civil time is behind (or earlier) than UTC so the zone designator will look like "03:00" Positive UTC offsets describe a time zone east of UTC±00:00, where the civil time is ahead (or later) than UTC so the zone designator will look like "+02:00"	"timeOffsetFromUTC" = "-02:30"	timeOffsetFromUTC: type: string description: An ISO8601-valid string representing the offset from UTC example: "+05:30" pattern: "±[hh]:[mm]"
Percentage	A decimal with a range from 0 to 100.00 that represents a ratio			"percentageComplete": "25.75"	

Address	General structure to capture the postal and physical address.			"address":{
				
				"addressLine1": "string",
				
				"addressLine2": "string",
				
				"addressLine3": "string",
				
				"addressType": "string",
				
				"buildingName": "string",
				
				"unitNumber": "string",
				
				"streetName": "string",
				
				"streetNumber": "string",
				
				"streetType": "string",
				
				"suburb": "string",
				
				"city": "string",
				
				"country": "string",
				
				"postalCode": "string",
				
				"state": "string",
				
				"status": "string"
				
				}	

Logical Operator	when applying a filter to a resource the default logical operator is AND. You can use this data type to change the behaviour to OR.		In the given example, the resource will return all customers who are having age = 75 OR name = abc OR country = AUS	GET /customers?$logicalOperator=OR & age = 75 & name = abc & country = AUS	$logicalOperator: type: string enum: - OR - AND

ERD fields	Reference data		The request should only contain the code, and response contains both code and desc	**<u>Request :</u>**
				
				"gender": {
				
				"code": "M"
				
				},
				
				"status": {
				
				"code": "A"
				
				}
				
				<u>Response:</u>
				
				"gender": {
				
				"code": "M",
				
				"description": "Male'
				
				},
				
				"status": {
				
				"code": "A",
				
				"description": "active	

				} 	

Error Object	General structure to capture the errors that are returned from an API			{
				"result": {
				"errors": [
				{
				"code": 0,
				"message": "string",
				"details": "string"
				}
				]
				}
				}
					Error:
					description: Represent an individual error.
					type: object
					properties:
					code:
					description: Error Code.
					type: integer
					format: int32
					message:
					description: Error Message
					type: string
					details:
					description: Detailed information about the error
					type: string

Common Identifier structures

The following identifiers and their structures for Westpac Mesh. They should always be used to ensure consistent identifiers across all APIs.

Entity	Composition	Example

accountId	accountNumber
	BSB
		"accountId": {
		"accountNumber": "123456",
		"BSB": "032000"
		}

financialTransactionCardId	cardNumber
	cardSuffix
		"financialTransactionCardId": {
		"cardNumber": "string",
		"cardSuffix": "string"
		}

customerId	"id": The string containing the customer identifier value
	"idScheme": The identification scheme used
		"customerId": {
		"id": "12345678901",
		"idScheme": "CustomerInternalId"
		}

Common Field Names

Field name	description	Type
emailAddress	email Id	String
applicationReferenceId	application Reference Number	String
phoneNumber	Phone Number	String
IPAddress	IPAddress	String
scheme	scheme Name (Master/Visa etc)	String

Field Naming Conventions

Data Type: Boolean

Booleans field names should start with 'is' or 'has'. It should be defined as type boolean. Eg: Yaml Syntax:

```
isInterestBearingAccount:
  description: Represent whether the account receives interest or not
  type: boolean
```

Data Type: Date/DateTime

All dateTime field must have 'dateTime' in the fieldName: Eg: arrangementEndDateTime.

All date fields must have 'date' in the fieldName: Eg: lastUpdatedDate.

Acronyms

All the acronyms must be in upper case. such as ATM, BSB, TFN etc

Abbreviations

It is generally preferrable to use full words in your field names over abbreviations. eg. 'previousComponent' instead of 'prevComp'.

Exceptions to this rule would be industry/domain standard abbreviations that are commonly used.

Defining Additional Information as Key / value pairs

In some cases , it is required to capture additional metadata of an entity in the form of an array of key/ value pairs.

Field Name	Description	Yaml Syntax

additionalMetadata	It is the additional details required . Eg: for creating package ,	"additionalMetadata": [
		{
		"name": "string",
		"value": "string"
		}
		]

Importing Common Data Types

Many of these common data types have predefined definitions that you can import into your API contract. The definitions can be found here:

https://schema.mesh.westpac.com.au/platform/contracts/0.12.0/common-types.yaml

You can import these as an external reference. For example:

```
  account:
    type: object
    properties:
      name:
        type: string
      balance:
        $ref: "https://schema.mesh.westpac.com.au/platform/contracts/0.12.0/common-types.yaml#
/definitions/Balance"
```

API Entity Model

There is a definition section in the swagger Spec that contains reusable definitions.

These definitions represent the core model of the API and contain only the properties that could be used within the scope of the resources defined for the API.

Examples: The customer resource for an API designed to expose only contact information would only define the email, telephone and/or postal addresses of the customer along with customer identifiers. It would not contain the IDV status and other demographic properties as they are not in the scope of the API.

Request and Response Definitions

These definitions describe the objects and parameters that are part of each resource's request and response. Request definitions include header parameters, query parameters, path parameters and body elements. Response definitions include header parameters (if applicable), body elements and HTTP response codes.

The body elements will contain the definitions from the API entity model.

Examples: Search customer, which is implemented as a POST operation, and thus the request has a body containing the search criteria.

Error Handling

Error Message Structure

```
{
        "result": {
                "errors": [
                        {
                                "code": 101,
                                "message": "Component not found",
                                "details": "Unable to find component for {componentId}"
                        }
                ]
        }
}
```

Error Property	Mandatory	Description
Code	YES	A specific API code used to identify the type of error. This is not the HTTP Code.
Message	YES	A consumer friendly description of the error with context if appropriate. I.e. 'Invalid customer number' (explicit error) or 'Missing mandatory field' (implicit - i.e. general due to provider not providing further information)
Details	NO	Detailed message of the error, usually derived from provider error message for business related errors. This could be pass through or filtered if sensitive information is returned. For technical errors this will be defined in the API implementation.

All errors should be returned in JSON format. Westpac Mesh APIs can respond with a single or multiple errors.

Error Classification

Error Cassification	HTTP Status Code	Error Code	Description
Business Error	4xx	1 - 9999	Related to data processed by the API, an error with consumer input
Technical Error	500	10000 or greater	Related to the environment, typically out of the consumer's control

Process for Mapping Errors

It is the responsibility of the API Designer to work with the Business Analyst to agree on which provider errors are 'business important'.

- Confirm whether errors being returned are actual errors and not warning or informational messages
- If no errors, return a HTTP response code 2xx
- If the error is bad data submitted by the consumer, return HTTP response code 4xx
- If the error was caused by failed processing on the server, return HTTP response code 5xx
- Once the appropriate HTTP response code is determined, identify the appropriate error code
- If none are appropriate, assign a new error code in the error block by allocating the next available number based on the error classification (business or technical)
- Provide an easily understood description of the error in the message element of the error block
- Map the provider's error message to the details element of the error block

When designing the errors in your API, put yourself in the shoes of a consumer and think about the types of errors you would want to know about.

It is important that the occurrence of an error is propagated up the call stack, but the error detail at each level must NOT be blindly propagated up the call stack. You can include provider error details from the downstream call in your API's specific error block to aid in troubleshooting.

Further reading: API Error Handling

Versioning and Backwards Compatability

Versioning

Semantic version should be honoured, and backwards compatibility should be ensured where possible.

- Patch version: Minor changes such as descriptions, formatting, additional enums, addition of new fields
- Minor version: Addition of new resources or operations
- Major version: A breaking change that typically requires a new API

Westpac Mesh APIs honour semantic versioning principles in that a new major version is not guaranteed to be backwardly compatible with a prior major version, a new minor version is backwardly compatible with a prior minor version, a new patch version is backwardly compatible with a prior patch version.

Westpac Mesh APIs supports both **URL versioning** and **Media type versioning.**

URL versioning is on by default in which include the major version number in the URI which allows side by side deployment of different major versions of an API

Media type versioning is available for Open API 3.0 and is a flexible solution for versioning endpoints without creating a new major version of an API.

- It allows versioning individual API endpoints when you need it.
- You can version input and output schemas independently.

Media type versioning allows the old endpoint to be marked as deprecated and a new endpoint to be created in the same version of the API. Only two endpoint versions are allowed before a major version needs to be created. When a new major version is created , any deprecated endpoints MUST be removed.

You should consider the pros and cons of each versioning strategy, and choose the one that suits your needs and preferences. You should also follow some best practices, such as minimizing breaking changes, documenting your API versions, and deprecating old versions gracefully.

The use of JSON instead of XML as a data format for APIs provides the opportunity to make many changes in a backwards compatible fashion. The table below illustrates some typical types of changes and whether they are backwardly compatible (non-breaking) or not. Note that the term 'operation' refers to the combination of HTTP verb and URI.

Type of change	Backward compatible
Add new mandatory request field (query,request body,header etc)to an existing operation.	No
Add new optional request field (query,request body,header) to an existing operation	Yes
Add new fields/objects to the response of an existing operation	Yes
Add new operation to an existing API	Yes
Rename or delete an existing field from an operation	No
Rename or delete an existing operation	No
Change cardinality of a field from 1 to n or n to 1	No
Add new http status code	No
Add new error code	Yes
Enhance enum(s) to add more values in request	Yes
Enhance enum(s) to add more values in response	No
Change mandatory request field(query,request body,header) to optional	Yes
Change data types in request /response	No
Change / Remove enum values in request / response	No
Change/ remove existing endpoint names	No
Change / remove existing http status codes / error codes	No
Change endpoint verb - example GET to POST	No

Note then when a new major version of an API is required, the new major version MUST include all operations and associated functionality from the prior version except where this functionality is deliberately being deprecated in the new version.

Paging, Sorting and Filtering

Status

ENDORSED

Problem Statement

How do we support paging, filtering and sorting of results returned by an API?

Approach

Paging

Paging is generally required when working with large datasets and allows the consumer to control how much data they wish to consume in a single call. Note. It is generally impractical to implement paging in an API if the provider (downstream system) does not natively support paging.

There are two main design decisions involved in implementing pagination for APIs:

a. How the client application will select and navigate to the data page it needs.
b. How to partition the response data that the API generates.

Where pagination is implemented there must be default values supported in addition to value overrides. Consistently use the parameter names “pageSize and pageKey” with default values for defining pages.

PageSize & PageKey with Defaults:

Use a simple pair of query parameters, which developers can use to specify pages. If they are not provided, then default values are applied.

/customers?pageSize=10

Followed by (for example):

/customers?pageSize=10&pageKey=10

Default values will depend on the likely use case and payload sizes. Also note, PageKey is a string, and PageSize is an integer.

Response data should include a paging block describing the total count, next page details, etc. Example of a paging block is shown below.

response with paging

```
"data":
{
...
}
"paging":
{
        "count": integer, //total number of items available
        "pageSize": integer, // number of items per page
        "nextPageKey": string, // key for start of next page
        "previousPageKey": string // key for start of previous page
}
```

Special case Scenarios

Case 1 : If a back end service returns data in a fixed page size, say 30 records per call, and first call will return the 30 records and then gives few extra information to indicate the next page set as below. The service also does not provide the total number of records.

```
"nextPageKey": {
  “Class": "string",
  "Type": "string",
  "RestraintAmount": "string",
  "StartDate": "2020-11-08",
  “EndDate": "2020-11-08",
}
```

In this case , we still can make use of our pagination pattern. Make the next page key an opaque key so the consumer is unaware of its structure. The simplest way to make this key opaque is to treat the key as a JSON object, and then base64 encode it and return this value as the paging

key. So, the consumer just sees a string which contains a bunch of alpha numeric characters. Using base64 encoding does not prevent the consumer from decoding it if they wish but we should not provide support for any consumer who tries to construct their own paging key. The API just needs to decode the paging key supplied by the caller, ensure that it reflects the JSON object then map the values to the provider interface.

The paging object in the response will be as below:

"paging": {

"pageSize": 30,

"nextPageKey": "encoded string"

}

Paging - Error Handling

Example

```
Default Page Key = 1
Default Page Size  = 10
total number of records available =  83

Scenario 1:  Get first 50 records
GET {{hostname}}/inf/bnkng/resrcitm/itappln/applnrepo/v1/applications?pageKey=1&pageSize=50

'The response should be as below. The previousPageKey is not returned as it is NULL for the first page
key.

"paging": {
        "count": 83,
        "pageSize": 50,
        "nextPageKey": "1"
    }

Scenario 2: GET  the second page.
GET {{hostname}}/inf/bnkng/resrcitm/itappln/applnrepo/v1/applications?pageKey=2&pageSize=50

The cal will return the remaining 33 records, and the paging block will look as below. Next Page key
will not be returned as it is the last page.
"paging": {
        "count": 83,
        "pageSize": 50,
        "previousPageKey": 1
    }

Scenario 3: GET the third page when third page does not exist.

GET {{hostname}}/inf/bnkng/resrcitm/itappln/applnrepo/v1/applications?pageKey=3&pageSize=50
The call should return the 400 Error clearly indicating the page requested is > the available pages.

Scenario 4: GET 500 records, and if the API can support max 100.
GET {{hostname}}/inf/bnkng/resrcitm/itappln/applnrepo/v1/applications?pageKey=1&pageSize=500
The call must return 400 Error with clear error message as 'Can't support the requested page size'
```

The calling application can then simply map the pageSize and nextPageKey/prevPageKey from the paging block in the response to the equivalent request parameters pageSize and pageKey for the subsequent call.

DO NOT TO RETURN nextPageKey/previousPageKey if it is null. As a general practice do not return any fields which are NULL in the response.

Note that the page keys are defined as strings to support both 'page numbers' and 'last record fetched' pagination styles.

Limiting result set (single page)

In the event that a provider is unable to support paging then the result set must be limited (capped) instead. All APIs which can potentially return large sets of data must support paging or limiting of result sets otherwise data may not be able to be returned to the API (provider timeout) or excessive amounts of data will be sent over the network. Before deciding to support limiting in an API the provider's ability to support paging must be thoroughly explored.

The pattern to support a request for limiting uses the same pageSize parameter as for paging but the pageKey is omitted. This is the same approach as when requesting the first page of a dataset. If the provider does not allow a pageSize to be specified but is hard coded to a specific value then the API should not expose a pageSize input parameter.

The response provided by the API must include the same paging object as for paging but the only fields which will be exposed are count and pageSize. It is critical that the count field is exposed so that the consumer is aware of the total number of rows available. However, if the provider cannot expose the total number of rows available then the count field should be omitted from the response.

Sorting

Sorting adds additional complexity so should only be implemented where there is a requirement, rather than by default. The preferred approach is that the downstream system be able to support sorting so that the API simply asks the downstream system to provide sorted results. The use of sorting with pagination can become complex if the downstream system does not support sorting. It can always be retrofitted without affecting backwards compatibility. Where sorting is justified, consistently use the following standard approach using query parameters.

Standard query parameter sort with comma-delimited list of fields to sort on, with + and - for ascending and descending, respectively.

Sort with Signed List:

/customers?sort=+age,+name

If the field is a structured resource, then use dot notation to identify the lower level attribute, e.g.

/customers?sort=+age,+name.surname

Filtering

Filtering is specific to the resource - it should not normally be implemented for all resource attributes as it may add unnecessary complexity. It can always be retrofitted without requiring a new version. There are valid arguments that for public APIs at least the amount of data returned should be able to be constrained to minimise data leaking.

There are 2 principal types of filtering that will be managed:

- **Limit Response Set:** Limit the records returned to match some value checks, e.g. age greater than 30, or postcode equal to 2000.
- **Limit Response Fields:** Limit the fields returned in the response to a set of preferred values. Should only be implemented for optional response fields.
    - **Using Views:** Limit the fields returned in the response to a set of preferred values using a named "view" comprising a set of fields. Use when the list of fields is large, but a subset of the whole and well known.

Limit Response Set:

Each field from the resource which can be used for filtering should be exposed as a query parameter named the same as the field. You should only expose those fields for which there is a clear requirement. The value to apply for the filter should be provided in the value portion of the query parameter key/value pair, as in:

GET /customers?firstName=John&lastName=Turner

Where multiple fields are supplied, as in the above example, the individual filters will be ANDed together so the result of the above query would be a list of customers with firstName of 'John' and lastName of 'Turner'.

By default, the comparison will be an exact match or equals, however if you require alternate semantics such as 'greater than' a specified value then an operator can optionally be supplied within the value string in the format of "operator:value". For example, to find customers who are older than 30 years of age you would write this filter:

GET /customers?age=gt:30

Shown below is the current list of operators, which will be extended over time as the need arises.

Operator	Operator syntax	Comparison value(s)
EQUALS (default)	eq	scalar value
NOT EQUAL	neq	scalar value
IN	in	comma separated list
NOT IN	nin	comma separated list
GREATER THAN	gt	scalar value

GREATER THAN OR EQUAL	ge	scalar value
LESS THAN	lt	scalar value
LESS THAN OR EQUAL	le	scalar value
BETWEEN	btn	comma separated list of 2 values

Examples:

```
?units=7           ; units equal 7
?units=gt:7        ; units greater than 7
?units=in:5,9,12   ; units equals 5 or 9 or 12

?createdDate =  "btn:2018-01-01,2018-06-30" – created Date between 01-01-2018 and 30-06-2018 (inclusive)

?settlementAmount =  "btn:200000,300000" - settlement Amount between 200000 and 300000 (inclusive)
```

Limit Response Fields:

Filtering of result columns (fields) will be implemented for optional fields using a standard query parameter structure as follows:

/customers/123456789?fields=name,emailAddress,mobileNumber

Views:

A variant of limiting response fields so that the list of fields is predefined and given a view name as an abstraction to avoid having to specify a long list of fields. The view can also expose fields not available in the base resource provided thtat they are derived from the base fields. This view name is supplied in the URI and appears similar to a subresource but contains a suffix of 'view' to identify it as being a view of the resource.

/customers/1234567/summaryView

Advanced (complex) filtering

When a complex set of filtering criteria are required (eg. cross field validation, object hierarchies) consider replacing the filter on the existing resource with POSTing to a new resource. As an example, a customer search needs to be able to match attributes for customers who are natural persons (individuals) and also for customers who are entities (organisations). When modelling the request message it would be natural for attributes which only relate to individuals to be contained within an individual structure and those attributes which only apply to organisations to be contained within an equivalent structure. A sample request body to search for an individual customer would look like the following.

customerSearch

```
POST /customerSearch HTTP/1.1
{
        "individual":
        {
                "firstName": "John",
                "middleName": "",
                "lastName": "Doe",
                "dateOfBirth": "1985-03-27",
                "gender": "male"
        }
}
```

Partial Success

Partial Success

It is possible for an API to respond with a partial success, with the response body containing both a data block and error block (and a pagination block if applicable).

The two use cases for a partial success are:

- The API accepts a list of items to operate on, and only some of the items were successfully processed. The list of successful items should be returned in the data block. The error block should A list of errors with a correlationKey referencing the items that failed.
- The API executes a process that contains multiple stages, but fails during one of the stages. In this case, a data block may not be returned (just a success HTTP code), but there must be an error block explaining the error.

Status

ENDORSED

- Problem Statement
- Approach
  - Example success response
  - Example failure response
  - Example partial success response
- Special case partial success response
- Partial Success with paging

Problem Statement

An API call can be partially successful for a number of reasons (one or more downstream systems are unavailable, poor cross-system data integrity, code defect, invalid user supplied input). A consumer of the API needs to be able to determine when there has been a partial success and react accordingly.

It is important to recognise that partial success is not a common use case and in most cases the API either succeeds or fails completely. A good example of a use case for partial success is an API which accepts a list of input objects, such as an account balance enquiry API which accepts a list of account numbers, and the processing of some of these objects might fail.

Approach

Current design patterns support success (2xx response codes) and failure (4xx and 5xx response codes).

Example success response

Success

```
HTTP/1.1 200 Ok
{
        data: {
                "field1" : "value1",
                "field2" : "value2",
                ...
        }
}
```

Example failure response

Failure

```
HTTP/1.1 400 Bad Request
{
        result: {
                errors: [{
                        "code": 45,
                "description": "invalid input field provided",
                "details": "Status must be one of Active, Inactive"
                }]
        }
}
```

Example partial success response

Partial Success

```
HTTP/1.1 200 Ok
{
    data: {
        "items": [ {
            "field1" : "value1",
            "field2" : "value2"
        }, {
            "field1" : "value1",
            "field2" : "value2"
        },
        ...
        ]
    }
    result: {
        errors: [{
            "code": 45,
            "description": "invalid account number provided",
            "details": "Account could not be found in XYZ system",
            "correlationKey" : {
                "accountNumber" : "12345678",
                "BSB" : "032733"
            }
        }]
    }
}
```

Note. The error object contained within the result object includes a new optional field 'correlationKey' which is intended to be used to relate the error to a specific object/entity, usually provided in the request to the API, which caused the error. Using the example of an account balance API which accepts a list of account numbers the correlationKey would contain the account number which caused the error.

Special case partial success response

There is a special case of partial success whereby there is no meaningful information to return for successfully processed items (eg. bulk update scenario) but we do want to return information about the failures, including the specific items which could not be processed.

If all processing completes successfully and there is no meaningful information to return then the API must return a HTTP response code 204 (No Content).

```
HTTP/1.1 204 No Content
```

If no processing is successful (ie. a complete failure) then the API must return a HTTP response code 500 (Internal Server Error) with associated 'result' object, as per our design standards.

If some processing is successful and some unsuccessful then the API must return a 200 (Ok) with the associated 'result' object but no 'data' object as there is no meaningful information to return.

```
HTTP/1.1 200 Ok
{
    result: {
        errors: [{
            "code": 15002,
            "description": "Failed to send SMS",
            "details": "Online banking registration completed successfully however failed to send SMS
due to {provider error}",
        }]
    }
}
```

Partial Success with paging

In some cases , we need to support partial success with paging.

The endpoint accepts a list of ids (some values) as input. The result of this would be a list for each of the supplied values. And for some of the values the records could be too many.

We can use paging with partial success in such situations .

The total number of records will be the total records across the list of values passed. Partial success does seem valid too as the values passed in may be invalid (ie. account does not exist) . The partial success block would only be returned on the first page of results.

```
HTTP/1.1 200 Ok
{
    data: {
        "items": [ {
            "field1" : "value1",
            "field2" : "value2"
        }, {
            "field1" : "value1",
            "field2" : "value2"
        },
        ...
        ]
    }
   paging:
    {
     "count": integer, //total number of items available across the list of id's passed
     "pageSize": integer, // number of items per page
     "nextPageKey": string, // key for start of next page
     "previousPageKey": string // key for start of previous page
    }
    result: {
        errors: [{
            "code": 45,
            "description": "invalid account number provided",
            "details": "Account could not be found in XYZ system",
            "correlationKey" : {
                "accountNumber" : "12345678",
                "BSB" : "032733"
            }
        }]
    }
}
```

Using Headers to avoid exposing sensitive information in the URI and query strings

PII should not be included in the Query String or path parameter where there is a risk that it may be stored in logs or on proxy servers. As such, where PII is included within a query string or path parameter, it is recommended that the string itself be moved to a header where it will not be cached or logged

Example Use cases

1. Sending Credit Card # in the path
2. Sending Customer email address / Phone Number as a query parameter

Example

```
Example 1 -  Sensitive Path Parameter

GET /creditCardArrangements/arrangementId
Headers:
    protected-arrangementId: 1234

Would be the equivalent of GET /creditCardArrangements/{arrangementId}

Example 2 -Sensitive Query Parameter

GET /customers
Headers:
    protected-phoneNumber: 1234
```

```
    protected-emailAddress: abc@email.com

Would be the equivalent of GET /customers?phoneNumber ="1234" && emailAddress="abc@email.com"
```

Further reading: Pattern 19: API Endpoint Design Patterns#Pattern19.3.5-UsingHeaderstoavoidexposingsensitiveinformationintheURIandquerystrings

Using Headers to avoid exposing sensitive information

Using Headers to avoid exposing sensitive information in the URI and query strings

PII should not be included in the Query String or path parameter where there is a risk that it may be stored in logs or on proxy servers. As such, where PII is included within a query string or path parameter, it is recommended that the string itself be moved to a header where it will not be cached or logged

Example Use cases

1. Sending Credit Card # in the path
2. Sending Customer email address / Phone Number as a query parameter

Example

```
Example 1 -  Sensitive Path Parameter

GET /creditCardArrangements/arrangementId
Headers:
    protected-arrangementId: 1234

Would be the equivalent of GET /creditCardArrangements/{arrangementId}

Example 2 -Sensitive Query Parameter

GET /customers
Headers:
    protected-phoneNumber: 1234
    protected-emailAddress: abc@email.com

Would be the equivalent of GET /customers?phoneNumber ="1234" && emailAddress="abc@email.com"
```

Further Reading: Pattern 19: API Endpoint Design Patterns#Pattern19.3.5a-UsingPOSTtoavoidexposingsensitiveinformationintheURIandquerystrings